To my parents,

whose sacrifices and love
made every page of this possible.

To my mentors,

who showed me that engineering
is as much about people as it is about machines.

Acknowledgements

I would like to express my sincere gratitude to all those who contributed to the completion of this work.

First and foremost, I would like to express my deepest appreciation to my academic supervisor, **Pr.K.ALLALI**, for his invaluable guidance, expertise, continuous support, and constructive feedback throughout this project. As both my supervisor and a member of the examination committee, his advice, encouragement, and thoughtful evaluation have been instrumental in shaping this work and enriching my academic journey.

I would also like to extend my sincere thanks to **Pr.N.MOUSSAID**, member of the examination committee, for accepting to evaluate this work and for the time, attention, and expertise devoted to its assessment. His valuable comments and recommendations are greatly appreciated.

I would like to express my heartfelt gratitude to my industrial supervisor, **Mr. Zhang ZHEQIAN**, as well as to the entire team at **Huawei Morocco** , for welcoming me into their professional environment and providing me with an enriching and rewarding internship experience.

I would also like to thank the faculty members of the **Department of Mathematics** at Faculty of Science and Technologies - Mohammedia for the solid academic foundation they provided throughout my studies.

Finally, I am forever grateful to my family and friends for their unwavering support, patience, and encouragement, which have been a constant source of strength throughout this journey.

Aya KIHAL
June 2026

Résumé

Ce projet de fin d'année, réalisé chez Huawei Maroc, présente la conception et la mise en œuvre d'une plateforme intelligente pour l'analyse de la migration de GPU vers NPU et l'aide à la décision en matière d'infrastructure d'IA.

Le projet répond au défi du déploiement de modèles d'IA dans des environnements matériels hétérogènes, particulièrement la transition des écosystèmes GPU basés sur CUDA vers les NPU Huawei Ascend. Le système proposé offre un cadre unifié pour analyser les charges de travail des modèles, évaluer la compatibilité matérielle, estimer la faisabilité de la migration et recommander des stratégies de déploiement optimales.

La plateforme repose sur une architecture modulaire composée de services d'analyse de modèles, de vérification de compatibilité, de benchmarking et de génération de recommandations. Elle introduit une couche d'abstraction matérielle permettant l'évaluation sur les infrastructures CPU, GPU et les futures infrastructures NPU.

La mise en œuvre suit les pratiques modernes de génie logiciel, incluant le déploiement conteneurisé et l'orchestration de services via API. Le benchmarking est utilisé comme mécanisme de validation pour soutenir les décisions de migration plutôt que comme un objectif primaire.

Le système démontre une approche structurée de la prise de décision en matière d'infrastructure d'IA en reliant les écosystèmes CUDA et Huawei Ascend, fournissant une base pour de futures optimisations et des stratégies de migration automatisées.

Mots-clés : migration GPU-NPU, Huawei Ascend, CUDA, infrastructure d'IA, abstraction matérielle, compatibilité de modèle, évaluation de performance

Abstract

This End-Of-Year engineering project, carried out at Huawei Morocco, presents the design and implementation of an intelligent platform for GPU to NPU migration analysis and AI infrastructure decision support.

The project addresses the challenge of deploying AI models across heterogeneous hardware environments, particularly the transition from CUDA-based GPU ecosystems to Huawei Ascend NPUs. The proposed system provides a unified framework to analyze model workloads, evaluate hardware compatibility, estimate migration feasibility, and recommend optimal deployment strategies.

The platform is based on a modular architecture composed of services for model analysis, compatibility checking, benchmarking, and recommendation generation. It introduces a hardware abstraction layer enabling evaluation across CPU, GPU, and future NPU infrastructures.

The implementation follows modern software engineering practices, including containerized deployment and API-based service orchestration. Benchmarking is used as a validation mechanism to support migration decisions rather than as a primary objective.

The system demonstrates a structured approach to AI infrastructure decision-making by bridging CUDA and Huawei Ascend ecosystems, providing a foundation for future optimization and automated migration strategies.

Keywords: GPU-NPU migration, Huawei Ascend, CUDA, AI infrastructure, hardware abstraction, model compatibility, performance evaluation

Contents

Acknowledgements **iv**

List of Figures **xvii**

List of Tables **xxi**

List of Acronyms **xxiii**

1 Introduction **2**

1.1 Introduction 2

1.2 Internship Context 3

2 General Context **5**

2.1 Huawei Group Overview 5

2.1.1 Introduction to Huawei 5

2.1.1.1 Strategic Positioning 6

2.1.2 Historical Background and Evolution 6

2.1.2.1 AI Computing Roadmap 7

2.1.3 Organisational Structure and Business Divisions 7

2.1.4 Technologies, Products, and Innovation 9

2.1.5 Research & Development 9

2.1.6 Huawei Morocco and Internship Context 9

2.2 Technical Context: AI Infrastructure Evolution 10

2.2.1 Introduction to AI Infrastructure 10

2.2.2 Evolution of Computing Architectures . . . 10
2.2.2.1 CPU Era (1940s–2006) . . . 10
2.2.2.2 GPU Era (2006–Present) . . . 11
2.2.2.3 Specialised AI Accelerators (NPUs, TPUs, ASICs) . . . 12
2.2.3 Increasing AI Computational Demands . . . 13
2.2.4 GPU Ecosystem Dominance . . . 14
2.2.5 Hardware Specialisation Trends . . . 15
2.2.6 Rise of Neural Processing Units . . . 15
2.3 Project Motivation . . . 16
2.3.1 Need for GPU → NPU Migration Guidance . . . 16
2.3.1.1 Structural Limitations Driving Migration . . . 16
2.3.1.2 Enterprise Migration Challenges . . . 16
2.3.2 Need for Compatibility Analysis . . . 16
2.3.2.1 Software Compatibility Dimensions . . . 17
2.3.2.2 Impact of Compatibility Problems . . . 18
2.3.3 Need for Hardware Transition Evaluation . . . 18
2.3.3.1 Performance Benchmarking . . . 18

3 Technological Background 20

3.1 GPU Computing Foundations . . . 20
3.1.1 CUDA Architecture Overview . . . 20
3.1.1.1 Streaming Multiprocessors . . . 21
3.1.1.2 SIMT Execution and Warp Model . . . 22
3.1.2 CUDA Execution Model . . . 22
3.1.2.1 Thread Hierarchy . . . 22
3.1.3 CUDA Memory Hierarchy . . . 23
3.1.4 CUDA Ecosystem and Software Stack . . . 24
3.2 NPU Computing Foundations . . . 25
3.2.1 NPU Architecture Principles . . . 25
3.2.2 Specialised Execution Hardware . . . 26

3.2.3 NPU Performance Metrics . . . 26
3.3 Huawei Ascend Ecosystem . . . 27
3.3.1 Ascend Hardware Architecture . . . 27
3.3.1.1 DaVinci Core Architecture . . . 28
3.3.1.2 Ascend 910 Chip Specifications . . . 28
3.3.2 CANN Runtime Architecture . . . 29
3.3.2.1 Layered Architecture . . . 29
3.3.2.2 Compiler Stack . . . 29
3.3.3 MindSpore Execution Framework . . . 29
3.3.3.1 Performance . . . 30
3.3.4 Ascend AI System Stack: End-to-End Integration . . . 30
3.4 AI Software Tools, Frameworks, and Infrastructure Stack . . . 31

4 State of the Art: Nemotron 3 Nano Case Study 34

4.1 Nemotron 3 Nano Overview . . . 34
4.1.1 Architecture . . . 34
4.1.2 Model Capabilities . . . 35
4.1.3 Model Family Positioning . . . 35
4.2 Complete Nemotron Training Lifecycle . . . 36
4.2.1 Data Preparation . . . 36
4.2.1.1 Pretraining Data Preparation . . . 36
4.2.1.2 SFT Data Preparation . . . 37
4.2.1.3 RL Data Preparation . . . 37
4.2.2 Pretraining Pipeline . . . 38
4.2.2.1 Execution Architecture . . . 38
4.2.3 Supervised Fine-Tuning Pipeline . . . 38
4.2.3.1 Training Architecture . . . 38
4.2.3.2 Model Parallelism Configuration . . . 39
4.2.4 Reinforcement Learning Pipeline . . . 39
4.2.4.1 Checkpoint Initialisation . . . 39

4.2.4.2 NeMo-Gym Environment Integration 40
4.2.5 Evaluation Pipeline 40
4.2.6 Inference and Deployment 41
4.3 Technical Stack Analysis 41
4.3.1 Megatron-Bridge 41
4.3.2 NeMo-RL 42
4.3.3 CUDA Dependency Analysis 42
4.3.4 Infrastructure Dependency Visualization 42
4.3.5 Experiment Tracking and Artifact Management 43
4.4 Consolidated Nemotron 3 Nano Architecture 43
4.4.1 End-to-End Architecture Description 43
4.4.2 Training Infrastructure Summary 44
4.4.3 Inference Infrastructure and Dependency Summary 45
4.4.4 Concluding Remarks 45

5 CUDA-to-CANN Migration: Implementation and Deployment 47
5.1 Use Case Definition and Dataset Construction 47
5.1.1 Dataset Blend 48
5.1.2 Dataset Preparation Pipeline 49
5.2 Architecture and Migration Strategy 49
5.2.1 Full Stack Migration Overview 49
5.2.2 Two-Track Migration Strategy 50
5.2.3 Cluster Configuration 51
5.2.4 Container Strategy 52
5.3 Critical Operator Migration 52
5.3.1 Mamba SSM Selective Scan Operator 52
5.3.1.1 Original CUDA Implementation 52
5.3.1.2 CANN Incompatibility 53
5.3.1.3 PyTorch Fallback Implementation 53
5.3.1.4 Kernel Injection Mechanism 53

5.3.1.5 Correctness Verification . . . 53
5.3.2 MoE Token Dispatch Operator . . . 53
5.3.2.1 Original CUDA Implementation . . . 54
5.3.2.2 PyTorch Fallback Implementation . . . 54
5.3.2.3 Load Balancing Loss . . . 54
5.3.2.4 Megatron-Bridge Dispatcher Patch . . . 54
5.3.3 Autocast and Mixed-Precision Patch . . . 54
5.4 Fine-Tuning Pipeline Implementation . . . 54
5.4.1 Patched Training Entry Point . . . 54
5.4.2 SFT Training Configuration . . . 55
5.4.3 Test Configuration . . . 55
5.4.4 W&B Artifact Lineage . . . 56
5.5 Inference and Deployment Pipeline . . . 57
5.5.1 Overview . . . 57
5.5.2 Model Conversion and Compilation . . . 57
5.5.3 Inference Server . . . 57
5.5.4 Performance Benchmarking . . . 58
5.6 Evaluation, Validation, and Results . . . 59
5.6.1 Evaluation Setup and Conditions . . . 59
5.6.2 Step 1 — Model Quality Evaluation . . . 59
5.6.2.1 Methodology . . . 59
5.6.2.2 Results . . . 60
5.6.2.3 Analysis . . . 61
5.6.3 Step 2 — Latency Benchmarking . . . 61
5.6.3.1 Methodology . . . 61
5.6.3.2 Analysis . . . 62
5.6.4 Step 3 — Throughput Benchmarking . . . 62
5.6.4.1 Methodology . . . 63
5.6.4.2 Analysis . . . 63

5.6.5 Step 4 — Energy Efficiency Measurement . . . 64
5.6.5.1 Methodology . . . 64
5.6.5.2 Analysis . . . 64
5.6.6 Consolidated Results and Migration Assessment . . . 65
5.6.6.1 Overall Assessment . . . 65

Conclusion **68**
Summary of Contributions . . . 68
Key Findings . . . 69
Limitations . . . 70
Perspectives and Future Work . . . 71

A Migration Codebase Reference **74**
A.1 Project File Inventory . . . 74
A.2 Dataset Formatters . . . 74
A.3 Mamba SSM Technical Reference . . . 75
A.4 MoE Routing Technical Reference . . . 77
A.5 CUDA-to-Ascend API Quick Reference . . . 77
A.6 Environment Variable Reference . . . 77
A.7 Inference Server Request/Response Schema . . . 77
A.8 Benchmark Test Prompts . . . 80

Bibliography & References **83**

Glossary **87**

List of Figures

2.1 Company Identity and Headquarters 6
2.2 Rotating Chair System Overview 8
2.3 Intel CPU Chip 11
2.4 NVIDIA GPU Chip 12
2.5 Google TPU Chip 13
2.6 GPU Market Share 2024 14

3.1 CUDA Architecture Overview 21
3.2 Kernel Execution Flow 23
3.3 CUDA Ecosystem Overview 24
3.4 Huawei AI Ecosystem 27
3.5 CANN Architecture Overview 29

4.1 End-to-End Nemotron AI training pipeline Overview 36
4.2 NVIDIA Infrastructure Dependency Diagram 43

5.1 CUDA-to-CANN Stack Mapping 49
5.2 Migration Strategy Overview 51
5.3 Inference and Deployment Pipeline Flow 57
5.4 Model Quality Evaluation for Base and Target Systems 60
5.5 Latency Benchnarking Comparaison for Base and Target Systems 62
5.6 Throughput Benchnarking for Base and Target Systems 63
5.7 Energy Efficiency Comparaison for Base and Target Systems 64

List of Tables

2.1 Huawei's key strategic priorities (2025) 6
2.2 Selected milestones in Huawei's corporate history 7
2.3 Huawei Ascend NPU product roadmap 7
2.4 Huawei business divisions and key products 8
2.5 Huawei annual R&D investment (selected years) 9
2.6 CPU architectural limitations for AI workloads 11
2.7 Core CUDA ecosystem components and their impact 12
2.8 Comparative specifications: GPU vs. NPU vs. ASIC 13
2.9 Growth in training compute for landmark models 14
2.10 Architectural comparison across computing paradigms 15
2.11 Structural limitations of GPU-centric AI infrastructure 17
2.12 AI framework compatibility status across hardware backends 17

3.1 CPU vs. GPU architectural characteristics 21
3.2 CUDA memory hierarchy: latency, bandwidth, and scope 24
3.3 Core CUDA libraries and their relevance to AI workloads 25
3.4 NPU on-chip memory hierarchy and access costs 26
3.5 Inference performance comparison: CPU, GPU, NPU 27
3.6 DaVinci core computational units 28
3.7 Huawei Ascend 910 key specifications 28
3.8 CANN compiler stack components 30
3.9 Full Ascend AI system stack 31

3.10 Complete AI software and infrastructure stack portability overview 32

4.1 Nemotron 3 Nano architectural specifications 35
4.2 Pretraining data preparation: inputs, outputs, and configuration 37
4.3 SFT data preparation: inputs, outputs, and key parameters 38
4.4 Pretraining configuration files 39
4.5 SFT stage artifact inputs and outputs (Nemotron Baseline) 39
4.6 RL stage key components and their roles 40
4.7 RL stage artifact inputs and outputs 40
4.8 Nemotron 3 Nano inference and deployment options 41
4.9 CUDA dependencies per execution stage 42
4.10 W&B artifact lineage across pipeline stages 44
4.11 Training infrastructure summary by stage 44
4.12 Inference infrastructure options 45

5.1 SFT dataset blend for the telecom support use case 48
5.2 Complete CUDA-to-CANN stack mapping 50
5.3 Mamba SSM implementation comparison 53
5.4 Ascend patch sequence in `train_sft_ascend.py` 55
5.5 SFT configuration parameter rationale 56
5.6 Artifact inputs and outputs for the Ascend SFT run 56
5.7 Expected inference performance — Ascend 910C, MindIE fp16, 4 NPUs, batch size 32 59
5.8 Evaluation infrastructure configuration 60
5.9 Standard benchmark results: CUDA baseline vs. Ascend target 61
5.10 Consolidated migration assessment 65

A.1 Complete file inventory — `nemotron-ascend-lifecycle/` 75
A.2 Dataset field-to-chat-format mapping 76
A.3 Mamba SSM tensor dimensions 76
A.4 CUDA API to torch_npu / AscendCL equivalent mapping 78

A.5 Ascend environment variables used in the migration 79

A.6 Benchmark test prompts by language and complexity 80

List of Acronyms

ACL Ascend Computing Language

API Application Programming Interface

ATC Ascend Tensor Compiler

CAGR Compound Annual Growth Rate

CANN Compute Architecture for Neural Networks

CBG Carrier Business Group

CLI Command Line Interface

CPU Central Processing Unit

CUDA Compute Unified Device Architecture

DSA Domain-Specific Architecture

EBG Enterprise Business Group

FFN Feed-Forward Network

GPU Graphics Processing Unit

GRPO Group Relative Policy Optimization

HBM High-Bandwidth Memory

HBM2 High-Bandwidth Memory Generation 2

HCCL Huawei Collective Communications Library

HCCS Huawei Cache Coherence System

HTTP Hypertext Transfer Protocol

ICT Information and Communications Technology

ISP Internet Service Provider

IT Information Technology

JSON JavaScript Object Notation

KV Key-Value

LLM Large Language Model

LoRA Low-Rank Adaptation

MAC Multiply-Accumulate

MENA Middle East and North Africa

MindIE Mind Inference Engine

MoE Mixture of Experts

MSA Modern Standard Arabic

MTE Memory Transfer Engine

NCCL NVIDIA Collective Communications Library

NPU Neural Processing Unit

P99 99th Percentile Latency

PEFT Parameter-Efficient Fine-Tuning

PTX Parallel Thread Execution

REST Representational State Transfer

RL Reinforcement Learning

ROUGE Recall-Oriented Understudy for Gisting Evaluation

RPS Requests Per Second

SCT Source Code Transformation

SFT Supervised Fine-Tuning

SIMT Single Instruction Multiple Thread

SLA Service Level Agreement

SM Streaming Multiprocessor

SSM State Space Model

TFLOPS Tera Floating-Point Operations Per Second

TOPS Tera Operations Per Second

TPU Tensor Processing Unit

vLLM Virtual Large Language Model

W&B Weights & Biases

CHAPTER 1

Introduction

1.1 Introduction

The rapid evolution of artificial intelligence has driven an unprecedented increase in the complexity and scale of machine learning models [1]. Modern applications — particularly large language models and multimodal systems — demand significant computational resources **kaplan2020scaling**, historically supplied by NVIDIA's CUDA-based GPU infrastructures. However, the current AI hardware landscape is undergoing a structural transformation, with specialised accelerators such as Neural Processing Units (NPUs) emerging as viable alternatives optimised for efficiency, energy consumption, and scalability [18].

This transition introduces substantial technical challenges: differences in execution models, framework compatibility, model conversion processes, and optimisation strategies make it increasingly difficult for organisations to determine whether a given AI workload can be efficiently migrated — and at what performance or cost implication.

This project addresses the problem by proposing an ***intelligent decision-support system*** that analyses AI workloads and evaluates their feasibility for migration across heterogeneous hardware environments. Specifically, the system focuses on the transition from CUDA-based GPU environments to Huawei's Ascend NPU ecosystem, providing structured insights into model compatibility, expected performance variation, and infrastructure suitability.

The proposed platform introduces a modular architecture separating workload analysis, hardware abstraction, migration evaluation, and recommendation generation. Rather than relying solely on traditional benchmarking, the system integrates benchmarking as a validation mechanism within a broader migration intelligence framework, enabling decision support that goes beyond performance measurement toward informed deployment choices.

The work is carried out within Huawei Morocco, leveraging the company's expertise in AI infrastructure, cloud computing, and hardware acceleration technologies [15]. Huawei's Ascend ecosystem — together with its software stack including CANN and MindSpore — provides the industrial foundation for exploring alternative AI execution environments and future-oriented deployment strategies.

1.2 Internship Context

The internship presented in this report was carried out within Huawei Morocco during the period April 1 – July 31, 2026. Work took place primarily at the Sidi Maarouf office in Casablanca, with professional activities also conducted between the Marina regional headquarters and the Rabat office.

The internship was conducted under the industrial supervision of Mr. Zhang ZHEQIAN, Data Center Solutions Department Director within the Cloud team at Huawei Morocco, and under the academic supervision of Pr. Karam ALLALI, Professor, Head of Department of Mathematics at Faculty of Sciences and Technology Mohammedia.

Throughout the internship, activities spanned two complementary dimensions. On the operational side, contributions were made to cloud infrastructure and migration processes within pre-sales and technical support missions of the Cloud team, providing practical exposure to enterprise IT environments, customer requirements analysis, infrastructure modernisation strategies, and cloud solution positioning within large-scale digital transformation initiatives.

On the research and engineering side, the internship centred on the design of a GPU-to-NPU migration intelligence and cross-hardware evaluation platform, focused on the comparison and analysis of AI workloads across the NVIDIA CUDA and Huawei Ascend ecosystems. This work evolved in a context where AI systems are increasingly dependent on specialised hardware accelerators, creating challenges related to workload portability, infrastructure selection, deployment optimisation, and ecosystem interoperability.

Work was occasionally conducted in collaboration with teams from the Enterprise Business Group (EBG), providing additional exposure to enterprise-oriented technological challenges and the operational dynamics of large-scale infrastructure solutions. This dual exposure to cloud operations and advanced AI systems research constituted a valuable opportunity to bridge theoretical engineering concepts with real-world technological and business constraints.

CHAPTER 2

General Context

2.1 Huawei Group Overview

2.1.1 Introduction to Huawei

Huawei Investment & Holding Co., Ltd. is a Chinese multinational corporation specialising in the research, development, manufacturing, and marketing of Information and Communications Technology (ICT) infrastructure and smart devices. Founded in 1987 by Ren Zhengfei in Shenzhen and wholly owned by its 169,054 employee-shareholders, Huawei operates across more than 170 countries with approximately 213,000 employees and serves over 3 billion people globally.

The company's overarching mission is to *"bring digital to every person, home, and organisation for a fully connected, intelligent world"*, emphasising AI as a universal technology comparable to water and electricity for all industries.

(a) Company Logo *(b) Company Headquarters*

Figure 2.1. *Company Identity and Headquarters*

2.1.1.1 Strategic Positioning

Huawei has positioned itself at the convergence of ICT infrastructure, AI computing, and cloud services under an "All Intelligence Strategy". Its core strategic priorities are summarised in Table 2.1.

Table 2.1. *Huawei's key strategic priorities (2025)*

Strategic Priority	Description
AI-First Approach	Full investment in Ascend NPUs, Kunpeng processors, and Pangu AI models
5G/5G-A Leadership	Supporting 230+ carriers worldwide; 5G-A users exceeded 60 million in 2025
Cloud Computing	Huawei Cloud deployed in 29 regions globally, serving 500+ scenarios across 30+ industries
Computing Ecosystem	Kunpeng: 3.8 M developers; Ascend: 4 M developers (2025)
Open Collaboration	9,800+ partners; 26,000+ industry solutions

2.1.2 Historical Background and Evolution

Huawei's 38-year evolution from a small telecommunications switch reseller to a global technology powerhouse is characterised by aggressive R&D investment, strategic international expansion, and remarkable resilience under geopolitical pressure. Table 2.2 presents the key milestones of this trajectory.

Table 2.2. *Selected milestones in Huawei's corporate history*

Year	Milestone
1987	Founded in Shenzhen with RMB 21,000 capital; initially reselling PBX switches
1993	Launched C&C08 program-controlled telephone switch — most powerful in China at the time
1997	First international contract (Hutchison Whampoa, Hong Kong); entered wireless GSM
2005	International revenue exceeded domestic sales for the first time
2010	First inclusion in Fortune Global 500 (revenue: $21.8B)
2012	Surpassed Ericsson to become world's largest telecommunications equipment manufacturer
2019	US Entity List designation; launched Ascend 910 (256 TFLOPS FP16 at 310W) and HarmonyOS
2024	Revenue CN¥862.1B ($118.1B), 22% year-on-year growth despite sanctions
2025	Ascend 910C mass-produced (SMIC N+2); Ascend ecosystem: 4M developers; ranked #83 in Fortune Global 500

2.1.2.1 AI Computing Roadmap

Huawei's AI computing strategy, detailed in Table 2.3, follows a one-year release cadence with the goal of doubling compute performance per generation.

Table 2.3. *Huawei Ascend NPU product roadmap*

Model	Year	Performance	Key Significance
Ascend 310	2018	Inference-focused	First AI chip in Ascend series
Ascend 910	Aug. 2019	256 TFLOPS (FP16)	World's most powerful AI processor at launch
Ascend 910C	Q1 2025	800 TFLOPS (FP16)	SMIC N+2 process; powers DeepSeek AI models
Ascend 950	2026	TBD	Two-variant next-generation chip
Ascend 960	2027	TBD	Continued AI chip roadmap
Ascend 970	2028	TBD	Targeting 4 zettaFLOPS (FP4) at system level

2.1.3 Organisational Structure and Business Divisions

Huawei operates under a unique employee-ownership governance model combining strategic board oversight with rotating executive leadership. The Rotating Chair system — in which the CEO role rotates on six-month terms among senior executives — is designed to

prevent concentration of authority and enhance organisational resilience under geopolitical pressure.

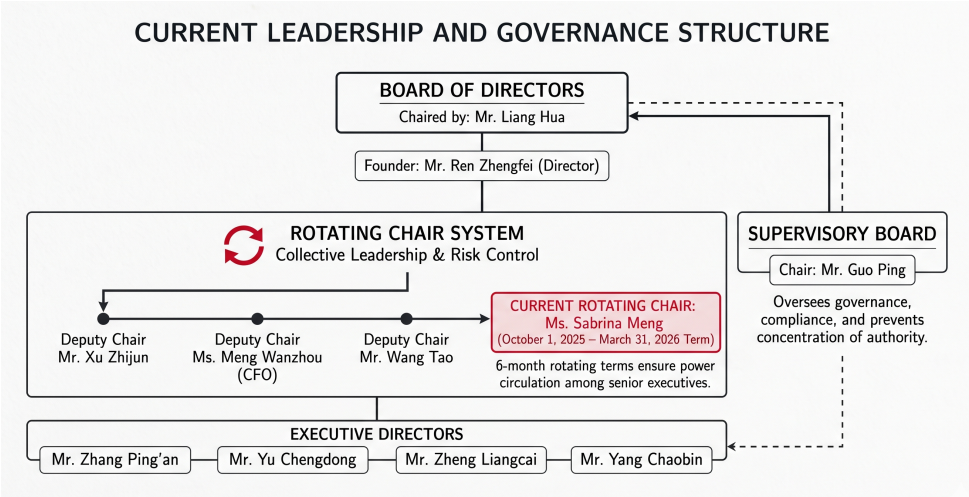

Figure 2.2. *Rotating Chair System Overview*

Huawei's operations are organised into four primary Business Groups plus specialised divisions, as summarised in Table 2.4.

Table 2.4. *Huawei business divisions and key products*

Division	Focus & Key Products
Carrier Business Group (CBG)	Telecommunications infrastructure: 5G/5G-A base stations, optical networks, core networks. Serves 230+ operators worldwide.
Enterprise Business Group (EBG)	Enterprise ICT: data centres (FusionCube, OceanStor), enterprise networking, private 5G, industry cloud solutions.
Consumer Business Group	Smart devices: Huawei smartphones (Pura, Mate series), HarmonyOS, wearables, tablets. 900M+ HarmonyOS devices (June 2024).
Huawei Cloud	Public cloud, AI services (Ascend-based instances), industry cloud. 29 regions globally, 500+ industry scenarios.
AI & Computing (HiSilicon)	Ascend NPUs, Kunpeng processors, MindSpore, CANN. Core of Huawei's post-sanctions technological independence strategy.
Digital Power	Solar inverters, energy storage, EV charging, data centre power systems.

2.1.4 Technologies, Products, and Innovation

Huawei's portfolio spans telecommunications infrastructure, AI computing, cloud services, consumer devices, and digital power solutions. Key product families directly relevant to this project are described below.

The Ascend NPU series forms the centrepiece of Huawei's AI computing strategy. CANN (Compute Architecture for Neural Networks) serves as Huawei's software abstraction layer analogous to NVIDIA's CUDA, while MindSpore is an open-source AI training and inference framework supporting Ascend-native execution. With 4 million developers in the Ascend ecosystem (2025), Huawei has established a viable alternative to the NVIDIA stack for Chinese AI infrastructure.

2.1.5 Research & Development

Huawei's R&D commitment is among the most intensive globally. In 2025, the company invested CNY 192.3 billion (US$27.5B), representing 21.8% of annual revenue — one of the highest R&D-to-revenue ratios in the global technology industry. Table 2.5 summarises recent investment levels.

Table 2.5. *Huawei annual R&D investment (selected years)*

Year	R&D Investment	% of Revenue	R&D Employees
2022	CNY 161.5B	25.1%	114,000
2024	CNY 170.0B	21.0%	117,150
2025	CNY 192.3B	21.8%	114,000

Huawei has ranked #1 globally in PCT patent applications for nine consecutive years (2014–2024), with 165,000+ active granted patents and US$630 million in patent licensing revenue in 2024.

2.1.6 Huawei Morocco and Internship Context

Huawei has maintained a presence in Morocco since 2007, establishing the country as the strategic hub for its Northern Africa operations — covering North, West, and Central Africa. Under the "In Morocco, For Morocco" philosophy, Huawei Morocco (Huawei Technologies Morocco S.a.r.l.) serves as the regional headquarters and actively contributes to Morocco's digital transformation under the Digital Morocco 2030 programme.

Huawei has partnered with Maroc Telecom, Orange Morocco, and Inwi for 4G/5G infrastructure deployment — Morocco officially entered the 5G era on November 7, 2025.

The company also supports 54 partner universities through the Huawei ICT Academy programme, targeting 1,100+ certified students by 2025–2026 across AI, cloud, 5G, and cybersecurity disciplines.

At the 2024 Digital Africa Summit in Marrakech, Huawei committed US$500M over five years to expand cloud coverage across Africa, reflecting Morocco's central role in its continental strategy.

2.2 Technical Context: AI Infrastructure Evolution

2.2.1 Introduction to AI Infrastructure

AI infrastructure encompasses the integrated hardware, software, and system-level components that enable the training, deployment, and operation of artificial intelligence models at scale. Unlike traditional IT infrastructure optimised for general-purpose data processing, AI infrastructure is specifically designed for the massive parallel computational demands of matrix operations, tensor computations, and neural network workloads.

The infrastructure stack is tightly coupled across four abstraction layers: hardware, system software, AI frameworks, and orchestration platforms.

AI infrastructure has become a strategic pillar for three interconnected reasons. First, the AI accelerator market is projected to grow from $45B in 2024 to $500B by 2030 (35% CAGR), reflecting its economic significance. Second, NVIDIA's 92% share of the data centre GPU market has spurred sovereign AI initiatives in the EU, China, and other regions. Third, AI computing power has been doubling approximately every 6.8 months — far exceeding Moore's Law — requiring continuous infrastructure innovation.

2.2.2 Evolution of Computing Architectures

2.2.2.1 CPU Era (1940s–2006)

From the 1940s through 2006, Central Processing Unit (CPU)s served as the sole computational backbone for AI research. While their complex control units excel at general-purpose branching logic, they face fundamental throughput and bandwidth limitations for modern parallel AI workloads, as summarised in Table 2.6.

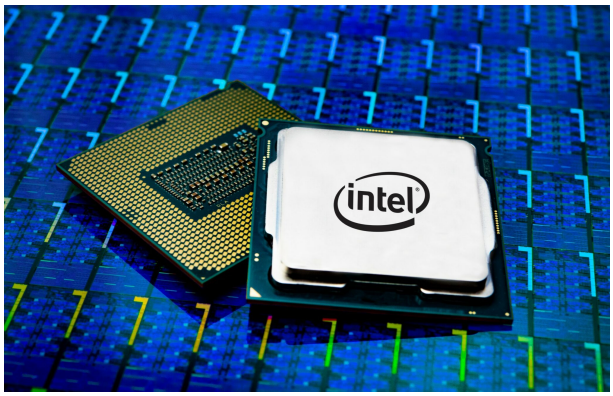

Figure 2.3. *Intel CPU Chip*

Table 2.6. *CPU architectural limitations for AI workloads*

Limitation	Technical Explanation and AI Impact
Sequential bottleneck	Typical CPUs have 4–96 cores optimised for sequential execution. Training ResNet-50 on a CPU takes ≈100× longer than on a GPU.
Limited memory bandwidth	DDR4/DDR5 bandwidth: 50–200 GB/s — insufficient for large-model weight streaming (requires 1–3 TB/s).
Poor FP throughput	Single-precision FP32: 0.5–2 TFLOPS; deep learning requires tens to hundreds of TFLOPS.

2.2.2.2 GPU Era (2006–Present)

The pivotal transition occurred in 2006 when NVIDIA introduced Compute Unified Device Architecture (CUDA), transforming GPUs from graphics accelerators into general-purpose parallel processors. The breakthrough moment came in 2012 when AlexNet, trained on GPUs, won ImageNet with 15.3% top-5 error (vs. 26.2% for the best CPU-based approach), demonstrating GPU-accelerated training 10–100× faster than CPUs.

Modern data centre GPUs achieve exceptional AI throughput through architectural features aligned with neural network workloads. The NVIDIA H100, for example, delivers 989 TFLOPS for FP8 sparse operations with 3,350 GB/s HBM3 memory bandwidth. The CUDA ecosystem amplified this hardware advantage through a suite of optimised libraries:

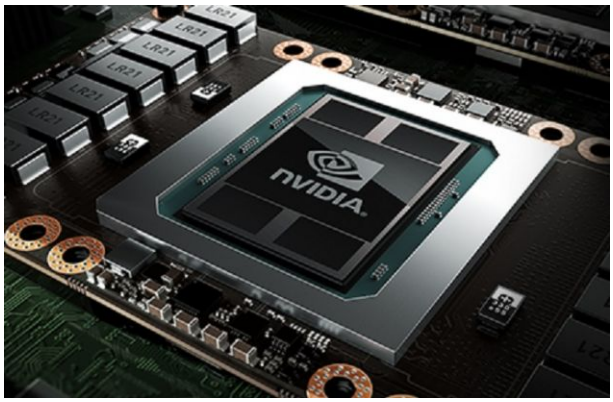

Figure 2.4. *NVIDIA GPU Chip*

Table 2.7. *Core CUDA ecosystem components and their impact*

Component	Function	Performance Impact
cuDNN	Deep neural network primitives	5–10× speedup for NN operations
cuBLAS	Linear algebra operations	Foundation for framework optimisations
TensorRT	Inference optimisation engine	3–5× faster inference
NCCL	Multi-GPU collective communications	Enables distributed training at scale

Despite their advantages, GPUs carry inherent limitations: the H100 consumes 700W TDP; training GPT-3 alone required ≈1,287 MWh; and a single DGX H100 system costs $300,000–$400,000. Furthermore, approximately 40% of GPU transistors are dedicated to graphics features unused in AI workloads.

2.2.2.3 Specialised AI Accelerators (NPUs, TPUs, ASICs)

Recognising GPU limitations, the industry transitioned toward Domain-Specific Architectures (DSAs). Google accelerated this shift with its Tensor Processing Unit (TPU) in 2015 — an ASIC designed exclusively for tensor computations. Table 2.8 compares key accelerator specifications.

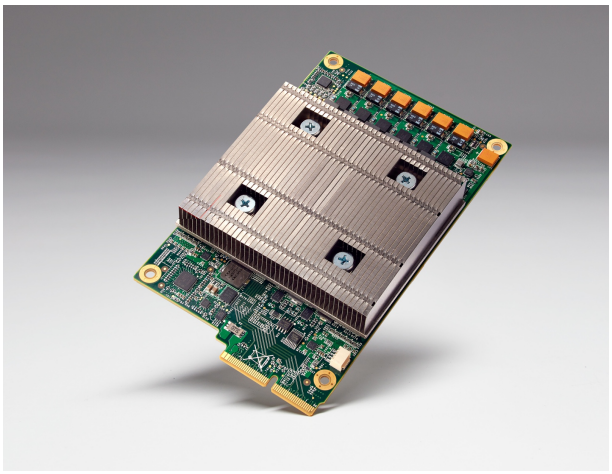

Figure 2.5. *Google TPU Chip*

Table 2.8. *Comparative specifications: GPU vs. NPU vs. ASIC*

Attribute	GPU (H100)	NPU (Ascend 910B)	ASIC (TPU v4)
Architecture	General parallel + graphics	Pure tensor accelerator	Fixed-function
Peak FP16	989 TFLOPS	256 TFLOPS	275 TFLOPS
Memory / BW	80 GB HBM3, 3,350 GB/s	64 GB HBM2, 1,200 GB/s	32 GB HBM2
TDP	700 W	310 W	170 W
Efficiency	≈1.5 TOPS/W	≈3–4 TOPS/W	≈5 TOPS/W
Ecosystem	CUDA (mature, proprietary)	CANN (Huawei)	XLA/JAX (Google)

2.2.3 Increasing AI Computational Demands

The exponential growth in AI compute demands has been the primary driver behind the hardware evolution described above. From AlexNet (2012) to GPT-4 (2023), parameter counts increased from 60 million to an estimated 1.7 trillion — a 28,000× increase. Training compute grew even more dramatically, from approximately 10^{16} FLOPs to beyond 10^{25} FLOPs — a billion-fold increase in just eleven years.

This multiplicative structure means that both model size and data scale jointly drive training cost. Table 2.9 illustrates the growth trajectory.

This growth rate — doubling roughly every 6.8 months — far exceeds historical Moore's Law (18–24 months per doubling) and creates an energy and cost crisis. Training GPT-3 consumed ≈1,287 MWh and cost ≈$4.6M in cloud compute; GPT-4 is estimated at

Table 2.9. *Growth in training compute for landmark models*

Model	Year	Parameters	Training FLOPs	Dataset
AlexNet	2012	60M	$\sim 10^{16}$	1.2M images
GPT-2	2019	1.5B	$\sim 10^{21}$	40 GB text
GPT-3	2020	175B	3.1×10^{23}	300B tokens
PaLM	2022	540B	6.5×10^{24}	780B tokens
GPT-4	2023	∼1.7T	$>10^{25}$	∼13T tokens

$63–78M. Data centre AI power demand is projected to more than double by 2030, potentially reaching 94.5 GW globally.

2.2.4 GPU Ecosystem Dominance

NVIDIA commands approximately 92% of the data centre GPU market as of 2024, a position built on three interlocking advantages: (1) the CUDA software stack with 4+ million developers and a decade-and-a-half head start; (2) continuous architectural innovation (Volta → Turing → Ampere → Hopper → Blackwell); and (3) a mature developer ecosystem where over 95% of papers at top AI conferences use NVIDIA hardware, creating reproducibility norms that reinforce adoption.

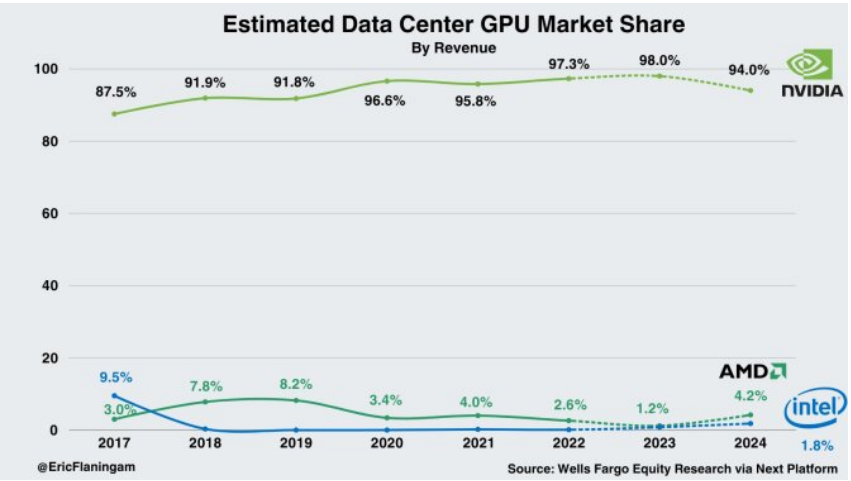

Figure 2.6. *GPU Market Share 2024*

This dominance, however, carries systemic risks. CUDA is proprietary and NVIDIA-exclusive, creating prohibitive migration costs for organisations with large CUDA codebases. H100 GPUs sell at $25,000–$40,000 per unit and carry 36–52 week delivery lead times. Training large models requires multi-node clusters drawing hundreds of megawatts. Communication overhead can consume 30–50% of training time in large clusters without specialised interconnects. US export restrictions on advanced AI chips to China

(2022–2023) further exposed the fragility of a single-vendor, single-geography ecosystem.

2.2.5 Hardware Specialisation Trends

The semiconductor industry is undergoing a fundamental shift from general-purpose to domain-specific architectures, driven by the convergence of three constraints: the end of Moore's Law scaling, energy efficiency boundaries, and the specialised computational patterns of AI workloads.

At process nodes below 5nm, quantum tunnelling effects and manufacturing complexity make further density increases prohibitively expensive. Meanwhile, AI workloads exhibit highly regular computational patterns — over 90% matrix multiplications and convolutions — that general-purpose processors cannot exploit efficiently.

Table 2.10. *Architectural comparison across computing paradigms*

Criterion	CPU	GPU	NPU/ASIC
Parallelism	Sequential + limited parallel	Thousands of cores	Systolic arrays, MAC units
Precision	FP32/FP64	FP16/FP32	INT8/FP16 optimised
Power (data centre)	65–350W	300–700W	10–310W
Efficiency	≈0.1 TOPS/W	≈2 TOPS/W	5–50+ TOPS/W
Best use case	Control logic, preprocessing	Training, batch inf.	Inference, edge AI

The AI computing landscape is bifurcating into cloud (training and large-scale batch inference) and edge (real-time, low-latency inference) domains, each with distinct hardware requirements.

2.2.6 Rise of Neural Processing Units

A Neural Processing Unit is a specialised microprocessor explicitly designed to accelerate AI and machine learning workloads by implementing the mathematical structure of neural networks directly in silicon. NPUs incorporate thousands of multiply-accumulate (MAC) units arranged in systolic array patterns, enabling massive parallel matrix multiplication — the fundamental operation in deep learning.

2.3 Project Motivation

The proliferation of generative AI and large-scale machine learning workloads has fundamentally transformed modern computing infrastructure. Organisations worldwide are investing heavily in AI-capable data centres, yet the prevailing reliance on GPU-dominated infrastructures has revealed critical limitations in cost, energy efficiency, supply chain resilience, and strategic autonomy. This project addresses the urgent industrial need for structured methodologies to evaluate, guide, and optimise the migration from GPU-centric architectures toward NPU-based alternatives.

2.3.1 Need for GPU → NPU Migration Guidance

Since the advent of deep learning, Graphics Processing Unit (GPU)s have established themselves as the de facto hardware foundation for AI workloads, with the market projected to reach USD 228.04B by 2030.

2.3.1.1 Structural Limitations Driving Migration

Despite this dominance, the GPU-centric paradigm is increasingly unsustainable. Table 2.11 summarises the five structural limitations that motivate the exploration of NPU-based alternatives.

NPUs address these limitations through domain-specific architectures. Recent benchmarks show NPUs delivering 30–63% higher energy efficiency than NVIDIA GPUs in quantised LLM inference, while operating at 10–30W per chip compared to 300–700W for data centre GPUs.

2.3.1.2 Enterprise Migration Challenges

Despite the promise of NPUs, enterprises face significant migration barriers: GPU-oriented ecosystems, unclear migration paths, limited NPU expertise, and workload heterogeneity (training vs. inference, latency vs. throughput, cloud vs. edge). The absence of structured guidance creates a systematic gap between the theoretical advantages of NPU adoption and practical deployment capability. This project directly addresses this gap.

2.3.2 Need for Compatibility Analysis

Compatibility analysis is a critical technical prerequisite during AI hardware transitions. The shift from GPU to NPU is not merely a hardware substitution but a fundamental

Table 2.11. *Structural limitations of GPU-centric AI infrastructure*

Limitation	Key Evidence
Energy consumption	A single GB200 NVL72 rack draws 120–132 kW vs. 10–15 kW for a typical enterprise rack; AI's energy needs projected at 21% of global electricity by 2030.
Infrastructure cost	Data centre construction costs rose from $7.7M to $10.7M per MW between 2020 and 2025 (7% CAGR).
Vendor lock-in	CUDA's dominance creates prohibitive migration barriers; multi-year supply agreements between NVIDIA and hyperscalers crowd out enterprise buyers.
Scalability for mid-size enterprises	Four GB200 racks require ≈0.5 MW of dedicated power plus liquid cooling infrastructure unavailable in most enterprise data centres.
Supply chain constraints	GPU delivery windows stretch 36–52 weeks; Blackwell-class hardware delayed to Q1 2027; NVIDIA consumes ≈60% of CoWoS packaging capacity.

reconfiguration of the entire software-hardware stack. A 2025 industry analysis found that 68% of production AI workloads contain at least one CUDA-dependent component requiring manual reimplementation for NPU deployment, rising to 85% for workloads using custom architectures.

2.3.2.1 Software Compatibility Dimensions

Table 2.12. *AI framework compatibility status across hardware backends*

Framework	Primary Backend	NPU Support Status	Migration Complexity
PyTorch	CUDA (NVIDIA)	Limited (requires CANN/MindSpore adaptation)	High
TensorFlow	CUDA (NVIDIA)	Partial (via Ascend TF plugin)	Medium-High
MindSpore	Ascend NPU	Native	Low
ONNX Runtime	Multi-arch	Moderate (provider-dependent)	Medium

CUDA creates dependency at multiple abstraction levels simultaneously: custom CUDA kernels, cuDNN/cuBLAS libraries, profiling tools (Nsight), and model checkpoint for-

mats with CUDA-specific metadata. Each layer must be validated independently during migration.

2.3.2.2 Impact of Compatibility Problems

Compatibility failures affect four dimensions of deployment success. **Feasibility**: severe incompatibilities can render deployment impossible without prohibitive re-engineering — cases include models with proprietary CUDA kernels and framework versions incompatible with NPU drivers. **Performance**: suboptimal operator implementations may reduce throughput by 30–50% and increase latency through memory layout mismatches. **Maintenance complexity**: organisations maintaining both GPU and NPU infrastructure must support two complete, evolving software stacks. **Cost**: industry estimates suggest 40–60% of total migration cost arises from software compatibility resolution, not hardware procurement.

Such migrations require not only code changes but framework-level retraining, model checkpoint conversion, and accuracy validation — illustrating the need for systematic compatibility analysis methodologies.

2.3.3 Need for Hardware Transition Evaluation

Hardware transition decisions represent multi-year infrastructure commitments that shape organisational AI capabilities. Systematic evaluation must span five complementary dimensions.

2.3.3.1 Performance Benchmarking

GPU and NPU performance profiles differ fundamentally across workload types. For inference at batch size 1, NPUs can achieve 3–10× lower latency than GPUs. For large-scale training, GPUs remain significantly superior. Benchmarks must be conducted under realistic production conditions — including data pipeline overhead, network latency, and memory contention — not idealised laboratory settings.

This project addresses the need for systematic methodologies, enabling organisations to make data-driven infrastructure decisions based on comprehensive technical and economic assessment rather than incomplete information or vendor marketing claims.

CHAPTER 3

Technological Background

3.1 GPU Computing Foundations

Graphics Processing Units have evolved from specialised graphics accelerators into the dominant computational paradigm for modern AI, enabling the training of deep neural networks at scales that would be infeasible on traditional CPU architectures. The transition began with NVIDIA's 2006 introduction of CUDA, which decoupled GPU computing from graphics APIs and enabled direct programmability for scientific and AI workloads. Since then, frameworks such as PyTorch and TensorFlow achieve their GPU performance through CUDA-accelerated libraries — cuBLAS, cuDNN — making GPU computing not merely advantageous but essential for contemporary AI systems.

The fundamental architectural distinction between CPUs and GPUs reflects different design philosophies: CPUs are latency-oriented, optimised for completing individual tasks quickly through complex control logic; GPUs are throughput-oriented, designed to process large data volumes in parallel by sacrificing single-thread performance for aggregate concurrency. This distinction is quantified in Table 3.1.

3.1.1 CUDA Architecture Overview

Compute Unified Device Architecture (CUDA) is NVIDIA's parallel computing platform and programming model, introduced in 2006 alongside the GeForce 8-series. Before CUDA, exploiting GPU compute required embedding algorithms inside graphics shader code — a severely limiting constraint. CUDA unified the shader architecture into programmable "CUDA cores" and exposed GPU parallelism through a C-like programming interface, establishing it as the *de facto* standard for GPU computing.

Table 3.1. CPU vs. GPU architectural characteristics

Feature	CPU	GPU
Design philosophy	Latency-oriented	Throughput-oriented
Core count	4–96 cores	Hundreds to thousands of cores
Control logic	Complex (branch pred., out-of-order)	Simplified (repetitive parallel ops)
Memory bandwidth	50–100 GB/s	200–3,350 GB/s
Thread parallelism	Dozens of threads	10,000+ concurrent threads
Cache hierarchy	Large L1/L2/L3	Smaller caches; compensated by bandwidth

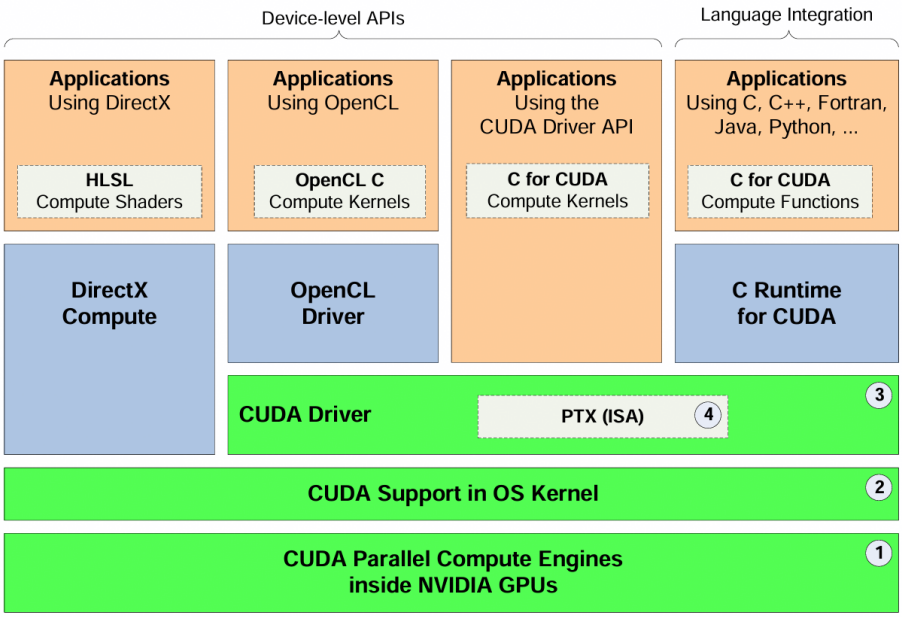

Figure 3.1. CUDA Architecture Overview

3.1.1.1 Streaming Multiprocessors

The Streaming Multiprocessor (SM) is the fundamental autonomous execution unit of any NVIDIA GPU. Each SM contains: CUDA cores (FP32/INT32 execution); Tensor Cores (matrix multiply-accumulate, introduced in Volta 2017); warp schedulers (typically 4 per SM); a large register file (32K–64K registers per SM enabling zero-overhead context switching); configurable shared memory / L1 cache (48–96 KB per SM); and special function units (SFUs) for transcendental operations.

For reference, the NVIDIA A100 contains 108 SMs with 64 CUDA cores each — totalling 6,912 cores — and can manage up to 2,048 threads per SM simultaneously, grouped into 64 warps of 32 threads.

3.1.1.2 SIMT Execution and Warp Model

CUDA implements the Single Instruction Multiple Thread (SIMT) model: groups of 32 consecutive threads called *warps* execute the same instruction in lock-step, each operating on different data. Key properties of warp execution are:

- All 32 threads in a warp execute the same instruction simultaneously.
- The warp scheduler switches between warps with zero overhead when one warp awaits memory — this is the primary latency-hiding mechanism.
- **Branch divergence**: when threads within a warp follow different execution paths, the warp serialises each path, potentially reducing throughput by up to 50%.
- Each SM can manage 64 warps (2,048 threads) but dispatches typically 4 warps per cycle through its warp schedulers.

3.1.2 CUDA Execution Model

3.1.2.1 Thread Hierarchy

CUDA organises threads into a two-level hierarchy. **Thread blocks** are groups of threads that share data through shared memory and can synchronise via barriers (`__syncthreads()`). Block sizes are multiples of 32 for warp alignment — typically 128, 256, or 512. A **grid** is the collection of all thread blocks executing a kernel. Each thread computes its unique global identifier as:

$$i = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x} \tag{3.1}$$

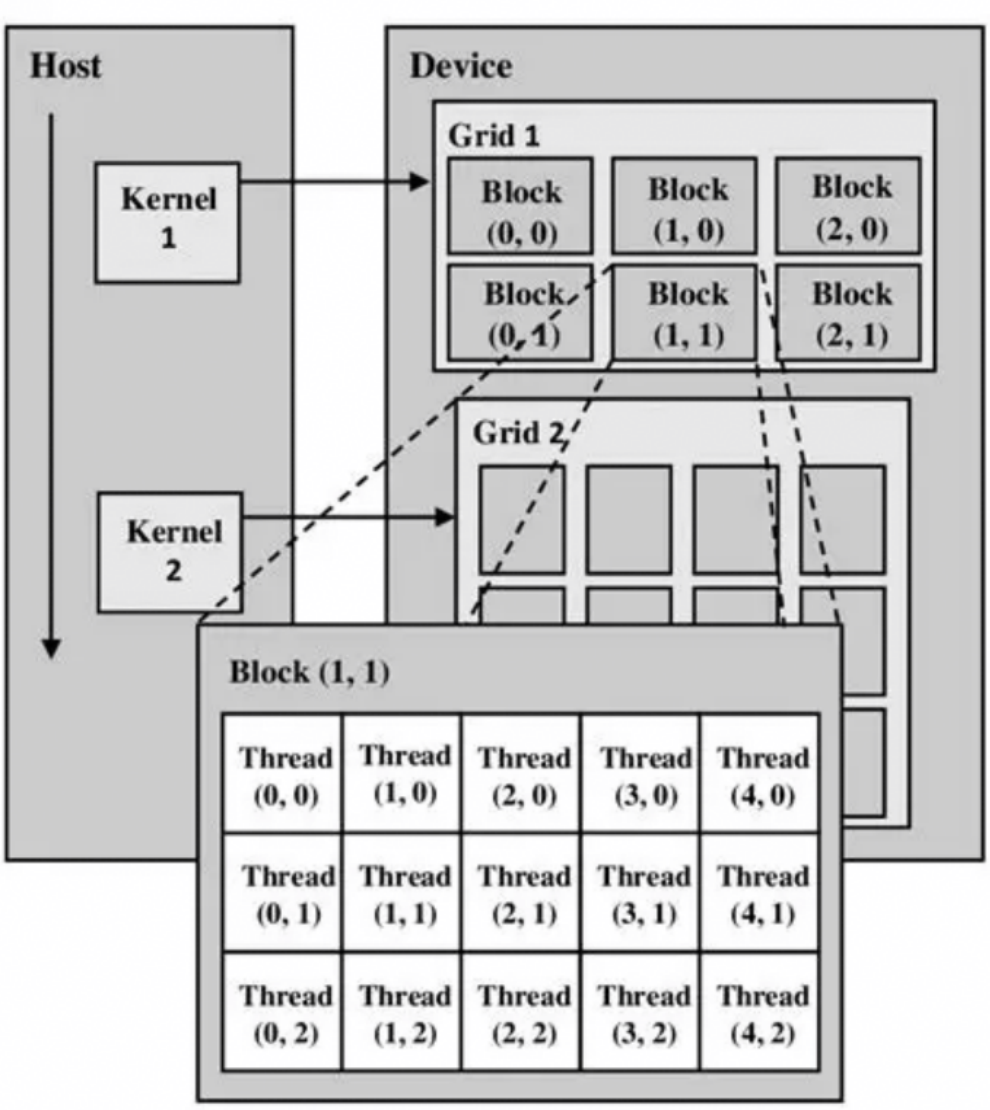

Figure 3.2. *Kernel Execution Flow*

3.1.3 CUDA Memory Hierarchy

CUDA exposes a multi-level memory hierarchy whose performance implications are central to kernel optimisation. Table 3.2 summarises the key characteristics of each level.

Three optimisation principles govern GPU memory performance. First, **memory coalescing**: consecutive threads should access consecutive addresses so the hardware merges accesses into fewer transactions — strided access patterns can reduce effective throughput by 10–50×. Second, **shared memory tiling**: caching frequently reused data in shared memory (20–30 cycle latency) avoids repeated 400–800 cycle global memory round-trips, which is critical for tiled matrix multiplication and convolution. Third, **register pressure**: excessive register usage reduces occupancy (active warps per SM), limiting the scheduler's ability to hide global memory latency — maintaining above 50% occupancy is typically the target.

Table 3.2. CUDA memory hierarchy: latency, bandwidth, and scope

Memory Type	Scope	Capacity	Latency	Bandwidth
Registers	Per-thread	∼255 regs	∼1 cycle	∼8 TB/s
Shared Memory	Per-block	48–96 KB/SM	20–30 cycles	>1 TB/s
L1 Cache	Per-SM	16–48 KB/SM	20–30 cycles	∼1 TB/s
L2 Cache	Per-GPU	4–6 MB	Tens of cycles	Moderate
Global Memory	Per-GPU	8–80 GB	400–800 cycles	200–3,350 GB/s
Constant Mem.	Per-GPU	64 KB	∼1 cycle (cached)	∼800 GB/s

3.1.4 CUDA Ecosystem and Software Stack

CUDA's dominance in AI infrastructure is as much a software phenomenon as a hardware one. The CUDA Toolkit provides a complete development environment: the `nvcc` compiler, profiling tools (Nsight Systems, Nsight Compute), and a suite of hand-tuned libraries summarised in Table 3.3.

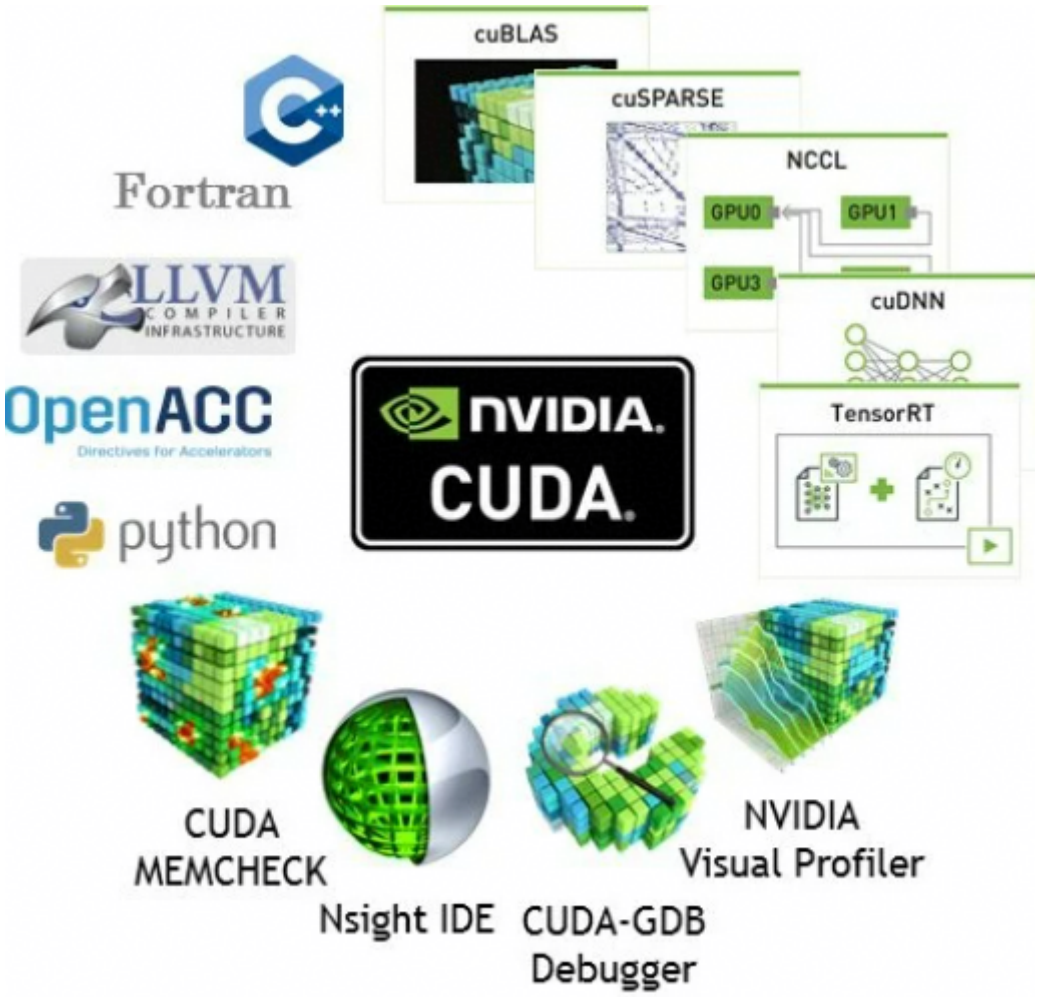

Figure 3.3. CUDA Ecosystem Overview

Table 3.3. *Core CUDA libraries and their relevance to AI workloads*

Library	Function	AI Relevance
cuDNN	Deep neural network primitives	Conv, pooling, normalisation, RNN; 5–10× speedup
cuBLAS	Basic linear algebra	Matrix multiplication for all linear layers
NCCL	Collective communications	Multi-GPU gradient synchronisation
TensorRT	Inference optimisation	Layer fusion, quantisation; 3–5× faster inference
CUTLASS	Templated GEMM kernels	Custom matrix operations for novel architectures

PyTorch and TensorFlow achieve GPU acceleration by dispatching operations to these libraries rather than implementing custom kernels. Over 95% of papers at top AI venues (NeurIPS, ICML, ICLR) use NVIDIA GPUs, creating reproducibility norms that reinforce adoption and make CUDA the path of least resistance for AI researchers and engineers.

This dominance, however, creates structural risks: CUDA is NVIDIA-exclusive; migration to ROCm, oneAPI, or CANN requires substantial re-engineering; and library optimisations are often CUDA-first — sometimes CUDA-only. These factors create the vendor lock-in dynamic that motivates the exploration of alternative ecosystems examined in subsequent chapters.

3.2 NPU Computing Foundations

3.2.1 NPU Architecture Principles

The most prevalent NPU computational substrate is the *systolic array*: a 2D grid of Processing Elements (PEs) arranged so that data flows rhythmically through the array like a heartbeat, with each element performing a MAC operation as data passes through. In a typical $N \times N$ configuration:

- Input activations stream west-to-east across rows.
- Weight matrices are pre-loaded north-to-south down columns.
- Partial sums accumulate southward and are written out at the bottom edge.

The key advantage is that each data element is read once from memory and reused across many computations as it flows through the grid, dramatically reducing memory bandwidth requirements relative to a GPU's scratchpad model.

NPU memory design is governed by the principle that physical distance from compute correlates directly with energy cost, with the four-level hierarchy summarised in Table 3.4.

Table 3.4. *NPU on-chip memory hierarchy and access costs*

Level	Type	Size	Access Cost	Purpose
PE Register File	Register	Bytes	1× (baseline)	Immediate MAC operands
PE Local Scratchpad	SRAM	KB	∼2×	Local PE data
Global Buffer	SRAM	16–24 MB	∼20×	Active weights and activations
Off-chip Memory	HBM/DRAM	GB	200–1,000×	Full model weights

3.2.2 Specialised Execution Hardware

A typical NPU execution pipeline consists of three specialised units working in concert. The **Matrix Engine** (or MAC Array, MAA) is a systolic array implementing the core GEMM operation, often fused with a bias accumulator. The **Vector Engine** handles element-wise operations: ReLU, sigmoid, tanh, layer normalisation. The **Dispatching Unit** (DSU) routes non-zero input feature maps to the MAA, enabling zero-skipping to accelerate sparse computations.

NPUs exploit *operator fusion* to eliminate intermediate memory round-trips. A Conv+Bias+ReLU sequence, for instance, executes as a single hardware pass — saving two global memory write-reads compared to three separate kernel launches as required on a GPU. For transformer models, the entire multi-head attention block (Q×K, Softmax, ×V) can be fused into a single dispatched operation.

3.2.3 NPU Performance Metrics

Tera Operations Per Second (TOPS) is the primary throughput metric for NPU compute capacity. The Huawei Ascend 910 achieves approximately 256 TFLOPS (FP16) and 512 TOPS in INT8 mode. Table 3.5 provides a quantitative comparison of inference performance across processor types.

Experimental benchmarks (KAIST) show that an NPU delivers 60% faster inference than a comparable GPU while consuming 44% less power for medium-complexity models. At high utilisation (MobileNetV2: 99.9%), NPUs achieve 143× better energy efficiency than CPU equivalents, with only 0.15–0.7% accuracy loss under INT8 quantisation.

Table 3.5. Inference performance comparison: CPU, GPU, NPU

Metric	CPU	GPU	NPU
Peak TOPS (INT8)	<0.001	30–624	256–512
FP16 TFLOPS	<0.1	100–989	256
Latency (ResNet-50)	50–100 ms	5–10 ms	1–3 ms
Throughput (img/s)	10–20	100–300	300–600
Energy efficiency	1–5 GFLOPS/W	10–30 GFLOPS/W	38–60 TOPS/W
Memory bandwidth	50–100 GB/s	200–3,350 GB/s	1,228 GB/s (Ascend 910)

3.3 Huawei Ascend Ecosystem

The Huawei Ascend ecosystem implements a full-stack AI computing infrastructure based on DaVinci architecture NPUs, the CANN runtime and compiler layer, the MindSpore framework, and the ModelArts cloud platform. This vertically integrated stack enables hardware-software co-design for optimal AI workload execution across device, edge, and cloud deployment scenarios — and constitutes the primary hardware target of this project.

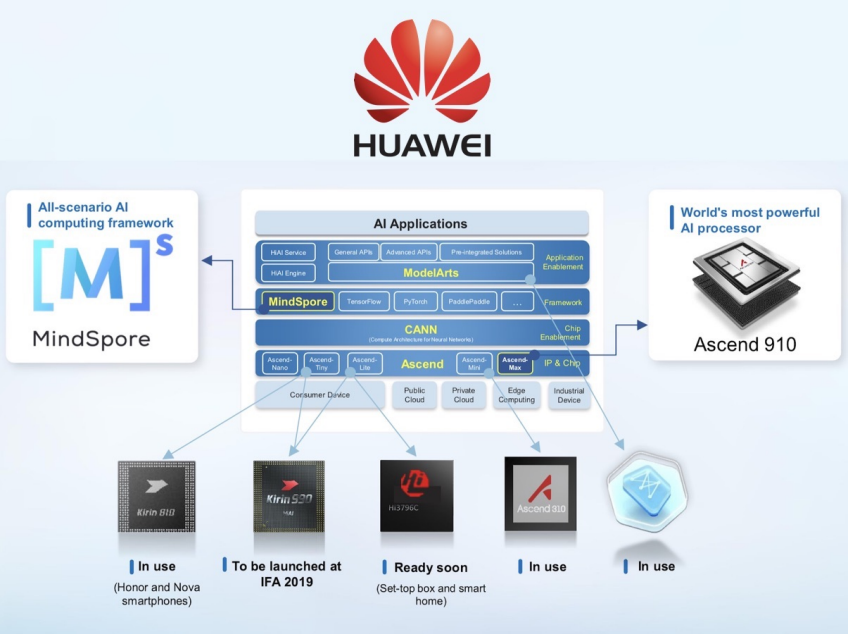

Figure 3.4. Huawei AI Ecosystem

3.3.1 Ascend Hardware Architecture

3.3.1.1 DaVinci Core Architecture

Huawei Ascend NPUs implement the DaVinci architecture, a 3D computational design consisting of three specialised processing units per core:

Table 3.6. *DaVinci core computational units*

Unit	Dimensionality	Function	Operations/Cycle
Cube Unit	3D ($16 \times 16 \times 16$)	Matrix multiply-accumulate	4,096 FP16 MACs + 8,192 INT8 MACs
Vector Unit	2D (2,048-bit)	Activation functions, element-wise	2,048-bit INT8/FP16/FP32
Scalar Unit	1D	Control flow, address generation	Full flexibility, branch logic

The Cube Unit performs the fundamental 3D matrix operation $D = A \times B + C$ where A, B, C, D are all 16×16 matrices, yielding 4,096 parallel FP16 MAC operations in a single cycle. The Vector Unit handles 2,048-bit wide activation and normalisation operations. The Scalar Unit manages control flow and address generation, enabling the Cube and Vector units to operate in pure dataflow mode.

3.3.1.2 Ascend 910 Chip Specifications

Table 3.7. *Huawei Ascend 910 key specifications*

Parameter	Specification
AI Cores	32 DaVinci cores
FP16 Performance	256 TFLOPS/s @ 1.0 GHz
INT8 Performance	512 TOPS
On-Chip SRAM	84 MB total (32 MB shared buffer)
Memory	32 GB HBM2 via 4 channels
Memory Bandwidth	1,228 GB/s off-die
Process Node	7nm+ EUV
TDP	300–350 W
Interconnects	PCIe 3.0 ×16, HCCS, 100G RoCE

The 32 DaVinci cores are organised in a mesh topology connected by the CHIE Network-on-Chip (NOC). Each core has independent L0 buffers (64 KB A + 64 KB B + 256 KB Unified), a 1 MB L1 buffer, and shares a 32 MB global buffer across all cores. Memory transfers between levels are managed by the dedicated Memory Transfer Engine (MTE).

3.3.2 CANN Runtime Architecture

Compute Architecture for Neural Networks (CANN) is Huawei's heterogeneous computing architecture providing the runtime, compiler, and operator library layer between AI frameworks and Ascend hardware — the direct analogue of NVIDIA's CUDA stack.

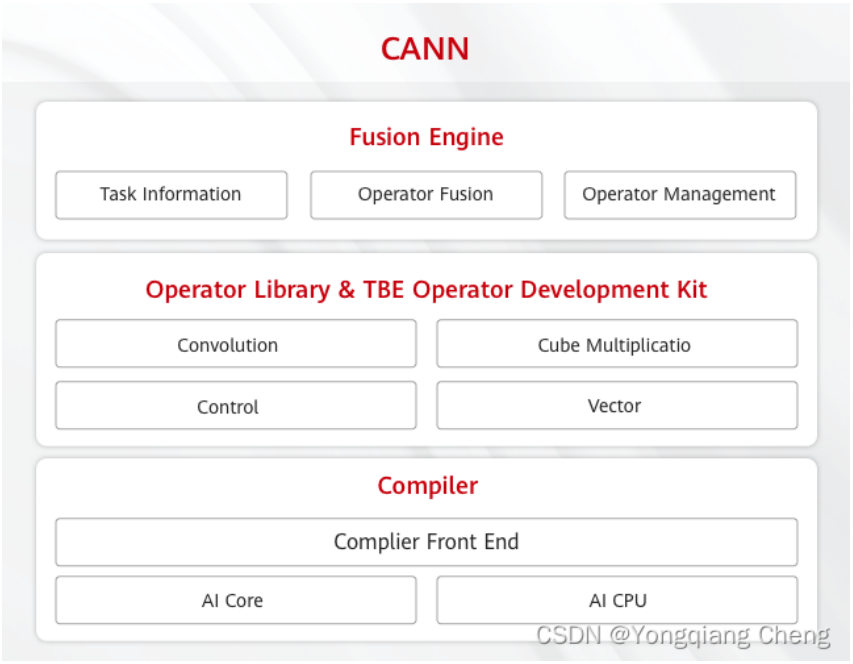

Figure 3.5. *CANN Architecture Overview*

3.3.2.1 Layered Architecture

CANN is organised into two abstraction layers. **L1 (CANN)** is the architecture-dependent layer providing hardware abstraction, task scheduling, and the AscendCL API. **L2 (Graph Engine)** is the architecture-independent optimisation layer performing operator fusion, cross-layer memory optimisation, and compute-communication parallelism.

AscendCL (Ascend Computing Language) is the unified programming interface analogous to CUDA's runtime API, covering device and context management, host-device memory transfers, model loading (from compiled `.om` files), and stream-based asynchronous execution.

3.3.2.2 Compiler Stack

3.3.3 MindSpore Execution Framework

MindSpore is Huawei's all-scenario AI framework supporting device-edge-cloud deployment. It employs Source Code Transformation (SCT) for automatic differentiation:

$$\text{Function} \xrightarrow{\text{SCT}} \text{IR} \xrightarrow{\text{Parse}} \text{Computational Graph} \tag{3.2}$$

Table 3.8. *CANN compiler stack components*

Component	Input → Output	Function
Graph Compiler	IR → Hardware executable	Converts framework IR to Ascend executable
TBE (Tensor Boost Engine)	Operator → kernel	Auto-schedules efficient operator compilation
OMG (Offline Model Generator)	Framework model → `.om`	Converts Caffe/TF/MindSpore to DaVinci offline model

MindSpore offers a unified graph mode that switches seamlessly between static execution (pre-compiled, production-optimised) and dynamic execution (immediate, research-friendly) — a design advantage over PyTorch's separation between eager and TorchScript modes.

In distributed training, MindSpore supports three parallel strategies (data, model, hybrid) with an automatic parallelism search that explores a large strategy space — contrasting with PyTorch's manual `DistributedDataParallel` configuration.

3.3.3.1 Performance

The MindSpore + Ascend 910 combination achieves 1,802 images/second on ResNet-50 V1.5 (ImageNet 2012) — approximately 2× the throughput of the mainstream TensorFlow + GPU baseline (965 img/s). This performance advantage stems from MindSpore's native CANN integration eliminating the translation overhead that external frameworks incur.

3.3.4 Ascend AI System Stack: End-to-End Integration

The four layers of the Ascend stack form a cohesive execution path from user-facing AI code to hardware compute:

An important engineering caveat: MindSpore-native models achieve ≈ 2× the throughput of PyTorch/TensorFlow running on Ascend via CANN, which incurs 10–20% translation overhead. For production systems targeting Ascend hardware, the native MindSpore path is strongly preferred; frameworks migrated from the CUDA ecosystem must weigh this performance cost against the effort of framework-level code changes.

Table 3.9. *Full Ascend AI system stack*

Layer	Component	Primary Function
L4	ModelArts	AI lifecycle management, distributed training orchestration, MLOps
L3	MindSpore	Computational graph execution, automatic differentiation, parallelism
L2	CANN	Graph optimisation, operator fusion, hardware abstraction, AscendCL API
L1	Ascend (DaVinci)	Tensor computation via Cube/Vector/Scalar units

3.4 AI Software Tools, Frameworks, and Infrastructure Stack

AI production systems are implemented as layered software stacks that separate model development, distributed training, optimisation, inference serving, orchestration, and infrastructure execution. In a GPU-to-NPU migration context, this stack defines where model code remains portable and where vendor-specific components introduce conversion, optimisation, and runtime dependencies.

The AI computing landscape is characterised by a fundamental asymmetry: while low-level infrastructure (Docker, Kubernetes) and MLOps tooling (W&B, MLflow) are largely hardware-agnostic, the performance-critical layers of model development and compilation are tightly coupled to vendor-specific ecosystems. Table 3.10 provides a mapping of equivalent tools across the NVIDIA and Huawei stacks and their portability.

The infrastructure layer defines how AI workloads are executed across local, cloud, and cluster environments. Containerisation via Docker and orchestration via Slurm or Kubernetes ensure that the environment remains consistent, which is critical for preserving reproducibility during cross-hardware migration studies. Environment drift is a primary source of performance loss, making clean infrastructure abstraction a prerequisite for any successful migration project.

Table 3.10. *Complete AI software and infrastructure stack portability overview*

Layer	NVIDIA Stack	Huawei Stack	Portability
Model dev.	PyTorch, HF Transformers, Megatron, DeepSpeed	MindSpore, MindFormers	Low
Distributed training	NCCL, DDP, FSDP	HCCL, MindSpore dist.	Medium
Optimisation	TensorRT, TensorRT-LLM, CUDA Graphs	CANN, ATC, ACL	None
Inference serving	vLLM, Triton, NIM	MindSpore Serving, ModelArts	Low
MLOps / tracking	W&B, MLflow, Slurm, Ray	W&B, MLflow, ModelArts	High
Infrastructure	Docker, Kubernetes, GitHub Actions	Docker, Kubernetes, GitHub Actions	Full

CHAPTER 4

State of the Art: Nemotron 3 Nano Case Study

4.1 Nemotron 3 Nano Overview

Nemotron 3 Nano is an open, efficient large language model developed by NVIDIA and released in 2025. It belongs to the Nemotron 3 model family — a suite of models spanning multiple capability tiers — and is positioned as a high-efficiency alternative to models with significantly larger active parameter counts. According to the repository root `README.md`, Nemotron 3 Nano *"achieves better or on-par accuracy than competitive models while having up to 3.3× higher inference throughput"* compared to models such as GPT-OSS-20B and Qwen3-30B-A3B.

The model is designed for **agentic reasoning** workloads: multi-step, multi-turn tasks requiring tool invocation, long-context comprehension, and structured output generation. Its primary use cases, as documented in the repository, include agentic pipelines with tool calling, extended context document analysis (up to 1M tokens), multilingual tasks across 15+ languages, and reasoning tasks with on/off reasoning-mode control.

4.1.1 Architecture

Nemotron 3 Nano implements a ***Hybrid Mamba-Transformer*** architecture combined with a ***sparse Mixture-of-Experts*** (MoE) design. This constitutes a departure from dense transformer-only architectures: the hybrid design interleaves Mamba state-space model layers with standard transformer attention layers, enabling efficient long-sequence processing without the quadratic complexity of full self-attention across all layers.

The sparse MoE component activates only a subset of expert networks per token during each

forward pass, decoupling total parameter count from per-token compute cost. Table 4.1 documents the architectural parameters extracted from the repository `README.md`.

Table 4.1. *Nemotron 3 Nano architectural specifications*

Parameter	Value
Total parameters	31.6B
Active parameters	3.6B (per forward pass)
Architecture type	Hybrid Mamba-Transformer with sparse MoE
Number of layers	32
Model dimension	3,008
Total routable experts	128
Activated experts	6 per token per MoE layer
Shared experts	2
Pretraining tokens	25 trillion
Context length	Up to 1M tokens
Training stages	3 (Pretrain → SFT → RL)

The 31.6B/3.6B parameter ratio means that each forward pass activates approximately 11.4% of the total parameters. This is the efficiency mechanism that enables inference throughput 3.3× higher than dense models of similar capability.

4.1.2 Model Capabilities

The following capabilities are documented in the repository `README.md` and constitute the target requirements for the full training pipeline described in subsequent sections.

4.1.3 Model Family Positioning

The Nemotron 3 family, as referenced in the repository, comprises three tiers: **Nano** (31.6B total / 3.6B active, optimised for throughput and edge/cloud inference), **Super** (mid-tier capability/cost balance), and **Ultra** (maximum capability, targeting frontier benchmark performance). Nano's design rationale is explicit in the repository: it trades total parameter count for inference efficiency by exploiting sparse expert activation, making it deployable on hardware configurations that cannot sustain dense models of equivalent benchmark performance. The training pipeline — documented across four stage directories — is shared in architecture across the family, with stage-specific configurations controlling model dimension and parallelism settings per tier.

4.2 Complete Nemotron Training Lifecycle

The Nemotron 3 Nano training pipeline is structured as a sequential three-stage process documented across four directories in the repository: *stage0_pretrain/*, *stage1_sft/*, *stage2_rl/*, and *stage3_eval/*. Each stage consumes the checkpoint artifact produced by the previous stage, forming a lineage chain tracked through Weights & Biases artifacts. The complete pipeline flow is illustrated in Figure 4.1.

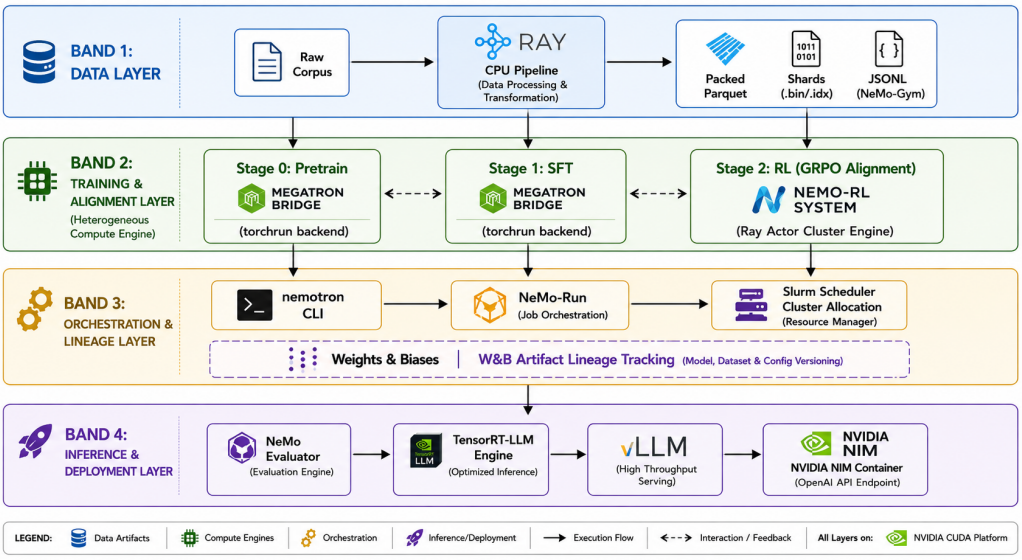

Figure 4.1. *End-to-End Nemotron AI training pipeline Overview*

4.2.1 Data Preparation

4.2.1.1 Pretraining Data Preparation

The pretraining data pipeline is implemented in *stage0_pretrain/data_prep.py*. Its purpose is to tokenize raw text corpora into Megatron's binary format (`.bin`/`.idx`) required by Megatron-Bridge's data loader.

The pipeline executes three sequential stages managed by the `cosmos_xenna.pipelines.v1` framework:

1. **PlanStage**: Reads the dataset blend definition from *config/data_prep/data_blend_raw.json*, performs deterministic work-item hashing, and generates per-shard `DatasetWorkItem` objects.
2. **DownloadStage**: Fetches each dataset shard from its source (Hugging Face Hub or local storage), one node per shard.

3. **BinIdxTokenizationStage**: Tokenizes documents using the tokenizer (`add_bos=False`, `add_eos=True`) and writes `.bin`/`.idx` file pairs.

Table 4.2. Pretraining data preparation: inputs, outputs, and configuration

Property	Detail
Entry point	`stage0_pretrain/data_prep.py`
Input format	Raw text datasets (HF Hub or local); blend defined in `config/data_prep/data_blend_raw.json`
Output format	Megatron `.bin`/`.idx` shards + `blend.json` per-split index
Sharding	128 output shards by default (1 if `sample` mode)
Tokenizer	`nvidia/NVIDIA-Nemotron-Nano-9B-v2`; `add_bos=False`, `add_eos=True`
Split output	`blend.json`: `{"train": [...], "valid": [...], "test": [...]}`
W&B artifact	`PretrainBlendsArtifact` (`nano3-pretrain-data`)
Execution mode	Ray (CPU-only nodes, `gpus_per_node=0`)

4.2.1.2 SFT Data Preparation

The SFT data pipeline (*`stage1_sft/data_prep.py`*) processes OpenAI-format chat datasets into packed Parquet shards with role-based loss masking. The pipeline applies four transformations in sequence:

1. **Chat template application**: The `nano3` chat template is applied to `messages` fields, producing role-labelled token chunks (system, user, assistant).
2. **Tokenisation with loss masking**: `input_ids` are generated with a `loss_mask` array where system and user tokens are masked to 0 and assistant tokens are set to 1, ensuring the model only learns to predict assistant responses.
3. **Sequence packing**: Sequences are packed up to `pack_size=4096` tokens using the `first_fit_shuffle` algorithm to maximise GPU utilisation during training.
4. **Parquet serialisation**: Packed sequences are written as `zstd`-compressed Parquet shards with row group size 1,000, compatible with Megatron-Bridge's `FinetuningDatasetConfig`.

4.2.1.3 RL Data Preparation

The RL data pipeline (*`stage2_rl/data_prep.py`*) converts preference datasets into JSONL format for consumption by NeMo-RL's NeMo-Gym interface. The `nemotron_rl` transform

Table 4.3. SFT data preparation: inputs, outputs, and key parameters

Property	Detail
Entry point	`stage1_sft/data_prep.py`
Input format	OpenAI chat format: `{"messages": [{"role", "content"}]}`
Output format	Packed Parquet shards; canonical split directories via symlinks
Loss mask	`loss_mask=0` for system/user; `loss_mask=1` for assistant
Pack size	4,096 tokens per packed sequence
Split ratios	`train=0.98`, `valid=0.01`, `test=0.01`
W&B artifact	`SFTDataArtifact` (`nano3/sft/data`)
Execution mode	Ray (CPU-only nodes, `gpus_per_node=0`)

extracts prompts from `responses_create_params.input` fields. Output is written to three split directories: `train/data.jsonl`, `val/data.jsonl`, `test/data.jsonl`, with a `manifest.json` tracking split paths and ratios.

4.2.2 Pretraining Pipeline

4.2.2.1 Execution Architecture

The pretraining stage is implemented in *`stage0_pretrain/train.py`* and uses Megatron-Bridge as the distributed training framework. The script is launched via `torchrun` across 2 nodes with 8 GPUs each (16 GPUs total). The configuration system uses OmegaConf YAML files with Hydra-style `key=value` CLI overrides, with two configurations provided in Table 4.4.

4.2.3 Supervised Fine-Tuning Pipeline

4.2.3.1 Training Architecture

The SFT stage (*`stage1_sft/train.py`*) uses Megatron-Bridge's `finetune()` function in place of `pretrain()`, with a `FinetuningDatasetConfig` built directly from packed Parquet shards rather than via the standard Hugging Face dataset loading path. This distinction is critical: the `_build_dataset_config()` function in `stage1_sft/train.py` constructs a `FinetuningDatasetConfig` with `PackedSequenceSpecs` pointing to the canonical split directories produced by data preparation, explicitly bypassing `HFDatasetConfig` to avoid unnecessary dataset downloads.

Table 4.4. Pretraining configuration files

File	Purpose	Key Parameters
`config/default.yaml`	Production	Recipe: `nemotron_3_nano_pretrain_config`; data from `nano3-pretrain-data:latest` artifact; checkpoint saved to `/nemo_run/nano3-pretrain-model`
`config/tiny.yaml`	Testing	Recipe: `qwen3_8b_pretrain_config`; `train_iters=1700`; `global_batch_size=32`; `lr_warmup_iters=32`

4.2.3.2 Model Parallelism Configuration

The production configuration specifies a four-way tensor parallelism layout (`tensor_model_parallel_size: 4`) with a hidden size of 1,344.

Table 4.5. SFT stage artifact inputs and outputs (Nemotron Baseline)

Direction	Artifact	Description
Input (model)	`nano3-pretrain-model:latest`	Pretrained Megatron checkpoint; `finetune: true` skips optimizer state loading
Input (data)	`nano3-sft-data:latest`	Packed Parquet shards + canonical split directories
Output	`nano3-sft-model:latest`	Fine-tuned Megatron checkpoint; saved every 20 iterations

4.2.4 Reinforcement Learning Pipeline

4.2.4.1 Checkpoint Initialisation

The RL stage handles two distinct checkpoint initialisation paths: (1) **HF conversion path**: the SFT Megatron checkpoint is converted to HuggingFace format via `megatron.bridge.AutoBridge.export_ckpt()`, enabling vLLM to load the fine-tuned weights directly for generation; (2) **fake step-0 path**: a `step_0/policy/weights/` directory structure is created by symlinking the Megatron checkpoint, allowing NeMo-RL's checkpointer to resume from it while vLLM continues using the base HF model for generation.

4.2.4.2 NeMo-Gym Environment Integration

NeMo-Gym provides the reward computation infrastructure. The `NemoGym` Ray actor is initialised with a `NemoGymConfig` containing the model name, vLLM server base URLs, and environment-specific configuration. A blocking `health_check` call ensures the actor is ready before training begins. The `task_to_env` mapping associates training tasks with their corresponding reward environments.

Table 4.6. *RL stage key components and their roles*

Component	Type	Role in Pipeline
`grpo_train()`	NeMo-RL function	Main training loop
`NemoGym`	Ray actor	Reward environment server
`policy_generation`	vLLM actor	Response generation
`policy`	Megatron model	Weight update (gradient computation)
`checkpointer`	NeMo-RL utility	Step-based checkpoint saving
`HCCL` / `NCCL`	Collective comm.	Gradient synchronisation

Table 4.7. *RL stage artifact inputs and outputs*

Direction	Artifact	Description
Input (model)	`nano3-sft-model:latest`	SFT checkpoint; converted to HF format or symlinked as step-0
Input (data)	`nano3-rl-data:latest`	JSONL files: `train/data.jsonl`, `val/data.jsonl`
Output	`nano3-rl-model`	Aligned model checkpoint; step-based saves via NeMo-RL checkpointer

4.2.5 Evaluation Pipeline

The evaluation stage resides in *`stage3_eval/`* and uses the NeMo Evaluator framework. Evaluation is configured via YAML benchmark configuration files that specify the target model artifact (`ModelArtifact-rl:latest`), benchmark suites, and metric collection parameters. The NeMo Evaluator deploys the model as an inference service, submits benchmark prompts, collects model outputs, and aggregates metrics.

4.2.6 Inference and Deployment

The repository root `README.md` and stage documentation reference three deployment paths for the final aligned model:

Table 4.8. *Nemotron 3 Nano inference and deployment options*

Engine	Role	Notes
TensorRT-LLM	Optimised inference	Model compiled to TRT engine; used in production NIM deployment
vLLM	Serving engine	Used during RL training for generation; also standalone serving path
NVIDIA NIM	Enterprise deployment	Container-based; wraps TensorRT-LLM runtime with OpenAI API

During the RL stage, vLLM is used directly as the generation backend within the `policy_generation` actor, confirming tight integration between the training and inference stacks.

4.3 Technical Stack Analysis

4.3.1 Megatron-Bridge

Megatron-Bridge is the distributed training framework used in Stages 0 and 1. It acts as an abstraction layer over Megatron-LM, providing: (1) a `ConfigContainer` dataclass for unified configuration of model architecture, training loop, optimiser, scheduler, logging, and checkpointing; (2) `pretrain()` and `finetune()` entry-point functions that encapsulate the Megatron training loop; and (3) a recipe system that maps named model architectures (e.g., `nemotron_3_nano_pretrain_config`) to fully-specified `ConfigContainer` instances.

The `create_omegaconf_dict_config()` utility converts the Python dataclass to an OmegaConf `DictConfig`, enabling merging of YAML file overrides and Hydra-style CLI overrides before converting back to the final `ConfigContainer` via `apply_overrides()`.

The `stage1_sft/config/default.yaml` file mounts specific Git commits of both Megatron-LM and Megatron-Bridge into the container at `/opt/megatron-lm` and `/opt/Megatron-Bridge`, confirming that production SFT uses pinned dependency versions rather than container image defaults.

4.3.2 NeMo-RL

NeMo-RL is the reinforcement learning framework exclusive to Stage 2. It provides the `grpo_train()` function, the `AllTaskProcessedDataset` data interface, the NeMo-Gym environment integration layer, and the `VirtualCluster` Ray initialisation utilities. Its container image (`nvcr.io/nvidia/nemo-rl:v0.4.0.nemotron_3_nano`) is distinct from the Megatron-Bridge NeMo container used in Stages 0 and 1, reflecting a separate dependency environment.

NeMo-RL's architecture within the RL stage involves four co-existing Ray actor types: the `policy` actor (Megatron-Bridge-backed weight update), the `policy_generation` actor (vLLM-backed generation), the `NemoGym` actor (reward computation), and the `cluster` virtual cluster coordinator. This actor decomposition allows generation and weight updates to proceed on separate GPU allocations within the same Ray cluster.

4.3.3 CUDA Dependency Analysis

Every execution stage in the Nemotron 3 Nano pipeline has a direct CUDA dependency. Table 4.9 documents this per stage based on repository evidence.

Table 4.9. *CUDA dependencies per execution stage*

Stage	CUDA Dependency	Functional Role
Data Prep	None (CPU-only)	Ray-based document tokenisation
Pretraining	`selective_scan_cuda`, cuBLAS, NCCL	Fused SSM kernels, matmul, distributed training
SFT	Megatron-Bridge, cuBLAS, NCCL	PyTorch-based fine-tuning acceleration
RL (GRPO)	vLLM, NeMo-RL, Ray, NCCL	High-throughput generation and weight updates
Evaluation	lm-eval, PyTorch	Parallel accuracy benchmarking
Inference	TensorRT-LLM, vLLM, Triton	GPU-optimised model serving

4.3.4 Infrastructure Dependency Visualization

The inter-dependency between the hardware, drivers, and frameworks is visualized in Figure 4.2.

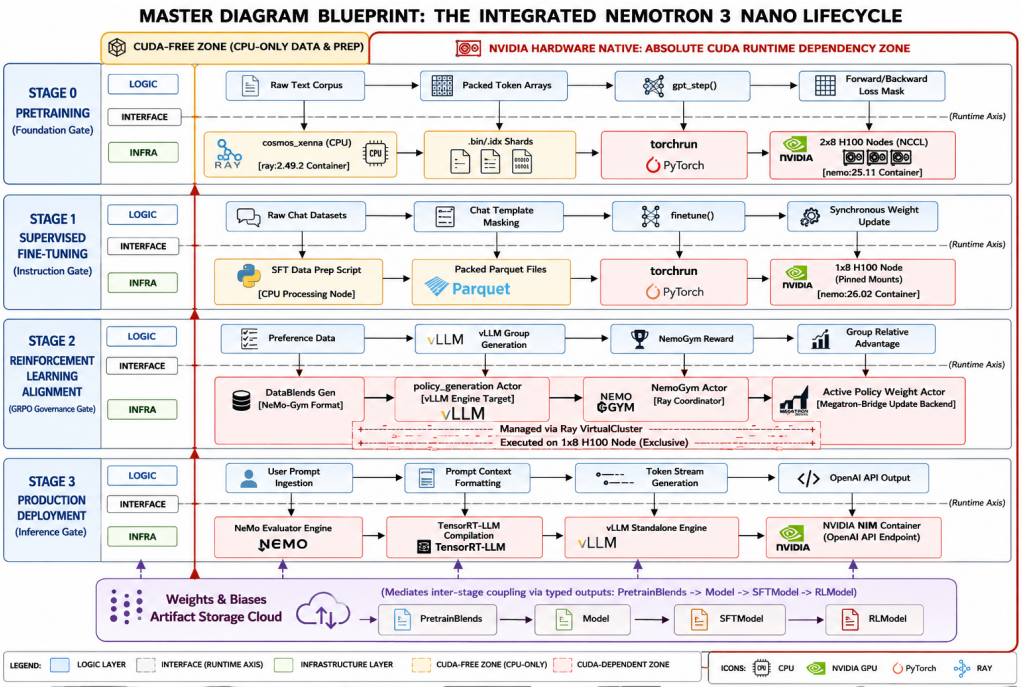

Figure 4.2. *NVIDIA Infrastructure Dependency Diagram*

4.3.5 Experiment Tracking and Artifact Management

Weights & Biases serves two distinct roles in the pipeline: experiment tracking and artifact lineage management. W&B is initialised in each stage to capture training metrics, configuration snapshots, and checkpoint logs, with several monkey-patches applied to ensure compatibility with Megatron-Bridge. The pipeline also uses W&B artifacts as the inter-stage handoff mechanism, creating an unbroken lineage graph from raw datasets to the final aligned model, as documented in Table 4.10.

4.4 Consolidated Nemotron 3 Nano Architecture

4.4.1 End-to-End Architecture Description

The Nemotron 3 Nano training and deployment system constitutes a vertically integrated NVIDIA-stack pipeline spanning data ingestion, distributed training, alignment, evaluation, and production inference. The architecture is characterised by three structural properties.

Stage isolation with artifact-mediated handoff. Each stage is an independently deployable unit communicating exclusively through W&B artifacts. **Framework heterogeneity across stages.** The pipeline uses Megatron-Bridge, NeMo-RL, and cosmos_xenna to address different training semantics. **Configuration-driven architecture selection.**

Table 4.10. W&B artifact lineage across pipeline stages

Stage	Artifact In	Artifact Out	Type
Pretrain data prep	Raw datasets	`nano3-pretrain-data:latest`	`PretrainBlendsArtifact`
Pretraining	`nano3-pretrain-dat latest`	`nano3-pretrain-mod latest`	`ModelArtifact`
SFT data prep	Raw chat datasets	`nano3-sft-data:latest`	`SFTDataArtifact`
SFT	`nano3-pretrain-mod latest` + `nano3-sft-data:latest`	`nano3-sft-model:latest`	`ModelArtifact`
RL data prep	Raw preference datasets	`nano3-rl-data:latest`	`DataBlendsArtifact`
RL (GRPO)	`nano3-sft-model:latest` + `nano3-rl-data:latest`	`nano3-rl-model`	`ModelArtifact`

The recipe system enables swapping model architectures via a single configuration field with no code changes.

4.4.2 Training Infrastructure Summary

Table 4.11. Training infrastructure summary by stage

Stage	Framework	Launch	Nodes	GPUs	Container
Data Prep	cosmos_xenna + Ray	ray	1	0	`anyscale/ray:2.49.2-py312`
Pretrain	Megatron-Bridge	torchrun	2	8/node	`nemo:25.11.nemotron_3_nano`
SFT	Megatron-Bridge	torchrun	1	8/node	`nemo:26.02.super.rc1`
RL (GRPO)	NeMo-RL + Ray	ray	1	8/node	`nemo-rl:v0.4.0.nemotron_3_nano`

4.4.3 Inference Infrastructure and Dependency Summary

Table 4.12. *Inference infrastructure options*

Engine	Integration Point	Deployment Context
vLLM	RL generation backend (`policy_generation` actor); standalone serving	Research and production serving
TensorRT-LLM	Post-training compilation of final checkpoint	Optimised production inference
NVIDIA NIM	Containerised packaging of TRT-LLM engine	Enterprise deployment; OpenAI-compatible API
NeMo Evaluator	Benchmark inference during Stage 3 evaluation	Accuracy measurement before deployment

4.4.4 Concluding Remarks

The Nemotron 3 Nano ecosystem implements a complete, production-grade LLM lifecycle spanning pretraining, supervised fine-tuning, reinforcement learning alignment, evaluation, and deployment. Each stage is implemented as a discrete, independently executable unit communicating through typed W&B artifacts. The training stack is exclusively NVIDIA-native: every GPU-accelerated stage depends on CUDA, cuDNN, NCCL, or TensorRT-LLM with no hardware-agnostic abstraction layer between the framework code and NVIDIA's accelerator stack. The data preparation pipeline, operating on CPU-only Ray nodes, is the sole stage with no direct CUDA dependency.

This architecture constitutes the reference NVIDIA-based ecosystem that will be analysed and used as the baseline for the migration study presented in the next chapter.

CHAPTER 5

CUDA-to-CANN Migration: Implementation and Deployment

5.1 Use Case Definition and Dataset Construction

The migration target for this project is a concrete, industrially grounded use case: a ***Multilingual Technical Support Assistant*** for MENA Telecom Infrastructure. The system is designed to assist enterprise clients of Huawei Morocco — telecom operators, ISPs, and enterprise IT teams — in resolving technical issues related to 5G infrastructure, Huawei network equipment, optical transport networks, and enterprise IT systems. Supported languages are English, Arabic, and French, reflecting the linguistic profile of Huawei Morocco's Northern Africa customer base.

This use case was selected for three concrete reasons. First, Nemotron 3 Nano's native multilingual support (15+ languages) and 1M-token context window align directly with the requirements of multi-document technical queries and multi-turn support conversations. Second, the 3.6B active parameter count fits within the 64 GB HBM2 of a single Ascend 910C, enabling single-node inference without tensor parallelism. Third, the domain-specificity of technical support provides clear, measurable evaluation criteria — keyword coverage, command correctness, and language accuracy — rather than the subjective metrics of general-purpose assistants.

The use case positions the fine-tuned model as the intelligence layer of a production support platform, with the inference server exposing an OpenAI-compatible REST API that can be integrated into ticketing systems, chat interfaces, and CLI tooling without modification.

5.1.1 Dataset Blend

The SFT dataset is constructed from four publicly available HuggingFace datasets, blended by weight to reflect the target language and domain distribution. Table 5.1 documents each source.

Table 5.1. *SFT dataset blend for the telecom support use case*

Dataset	HuggingFace Path	Lang.	Size	Weight
Salesforce DialogStudio	`Salesforce/dialogstudio`	EN	∼50K	0.40
Arabic Alpaca	`kunishou/ArabicAlpaca`	AR	∼52K	0.30
Open-Platypus	`garage-bAInd/Open-Platypus`	EN/FR	∼25K	0.20
OPUS-100 (fr-ar)	`Helsinki-NLP/opus-100`	FR/AR	∼10K	0.10
Total			∼137K	1.00

Each dataset undergoes a distinct formatting transformation implemented in *`data/prepare_datasets.py`*. The **DialogStudio TechSupport** split contains multi-turn technical support conversations in OpenAI dialog format; each dialog is parsed by iterating over `roles` fields and mapping `USR` tokens to `user` turns and `SYS` tokens to `assistant` turns. The **Arabic Alpaca** dataset uses an instruction-input-output structure; the `instruction` and optional `input` fields are concatenated into the `user` turn, and the `output` becomes the `assistant` turn. The **Open-Platypus** dataset is filtered to retain only samples containing technical keywords (network, protocol, server, system, réseau, etc.) before formatting. The **OPUS-100 fr-ar** translation pairs are converted into synthetic bilingual QA samples where a French technical sentence becomes the user query and the Arabic translation becomes the assistant response, filtered to retain only telecom-relevant vocabulary.

All samples are formatted to a unified OpenAI chat JSON schema with system, user, and assistant roles, as described in Chapter 3.

Three language-specific system prompts are defined in *`data/prepare_datasets.py`* and injected based on dataset source:

- `SYSTEM_PROMPT_EN` — English prompt positioning the model as a Huawei telecom support specialist
- `SYSTEM_PROMPT_AR` — Arabic prompt with equivalent framing in MSA
- `SYSTEM_PROMPT_FR` — French prompt for French-language client interactions

During inference, the server automatically detects the language of the incoming user query (via Unicode Arabic character ratio and French lexical markers) and injects the appropriate

system prompt, ensuring consistent role framing without requiring the client application to supply a system prompt.

5.1.2 Dataset Preparation Pipeline

The dataset preparation executes in two stages. In the first stage, raw datasets are downloaded and formatted to `.jsonl` using *`data/prepare_datasets.py`*.

The second stage runs the Nemotron SFT data pipeline on the formatted files to apply the chat template and produce packed Parquet shards.

5.2 Architecture and Migration Strategy

5.2.1 Full Stack Migration Overview

The migration replaces every CUDA-dependent component in the Nemotron 3 Nano pipeline with a CANN-compatible equivalent, while preserving the components that are hardware-agnostic. Table 5.2 documents the complete mapping.

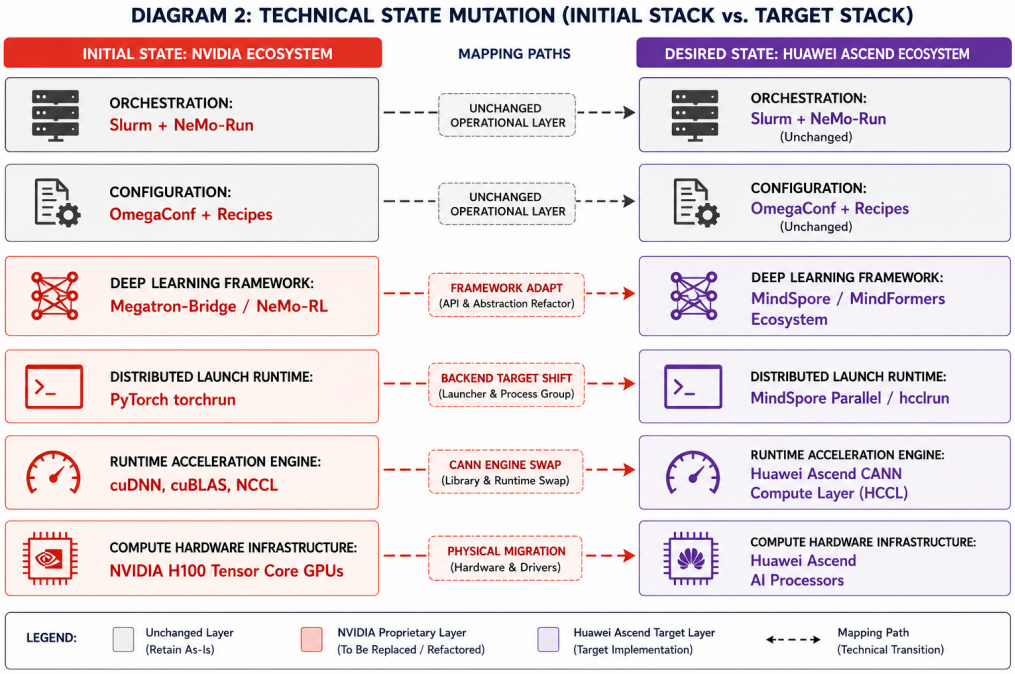

Figure 5.1. *CUDA-to-CANN Stack Mapping*

Table 5.2. Complete CUDA-to-CANN stack mapping

Layer	CUDA (Original)	CANN (Target)	Change
Hardware	NVIDIA H100 SXM5	Huawei Ascend 910C	Infrastructure
Compute runtime	CUDA + cuDNN + cuBLAS	CANN AscendCL + CANN NN Library	Abstracted via `torch_npu`
Distributed comm.	NCCL	HCCL	Backend string change
Training framework	Megatron-Bridge + torchrun	Megatron-Bridge + torch_npu + torchrun	Patched entry point
Mamba SSM kernel	selective_scan_cuda	ops/mamba_ascend.py	New file
MoE dispatch kernel	Fused CUDA gather/scatter	ops/moe_routing_ascend.	New file
Mixed precision	torch.cuda.amp.autocast	torch.npu.amp.autocast	10-line patch
Data preparation	Ray CPU (no GPU)	Ray CPU (unchanged)	None
Inference engine	TensorRT-LLM / vLLM	MindIE / vLLM-Ascend	New compilation step
Inference server	Triton / NIM	inference/serve.py	New file
Observability	Weights & Biases	Weights & Biases (unchanged)	None
Orchestration	NeMo-Run + Slurm	NeMo-Run + Slurm + env.toml Ascend profile	Config addition
Containerisation	NVIDIA NeMo images	docker/Dockerfile.sft + .inference	New Dockerfiles

5.2.2 Two-Track Migration Strategy

The migration follows a ***two-track parallel-path strategy*** that separates correctness validation from production optimisation.

Track A — PyTorch-NPU (validation track). The `torch_npu` package registers Ascend as a PyTorch device type (`"npu"`) and provides the `transfer_to_npu()` monkey-patch that intercepts all `.cuda()` calls across the entire Python process, including deep inside Megatron-Bridge. This allows Megatron-Bridge to run on Ascend with minimal code changes. The Mamba SSM and MoE operators are replaced with pure-PyTorch fallback implementations that are correct on any device but 3–8× slower than the fused CUDA originals. Track A's goal is to establish numerical correctness — a loss curve that matches the CUDA baseline within 2% — before any performance optimisation.

Track B — MindSpore-native (production track). Full reimplementation of model definition and training loops in MindSpore with MindFormers, achieving maximum performance through CANN-native graph execution with automatic operator fusion. Track B is the production path but requires significantly more engineering effort. It is developed in parallel with Track A, using Track A's loss curves as the numerical correctness reference.

This project implements **Track A in full** — the complete, runnable fine-tuning and inference pipeline — which constitutes the primary deliverable. Track B is identified as future work, with the correctness baseline established by Track A serving as its acceptance reference.

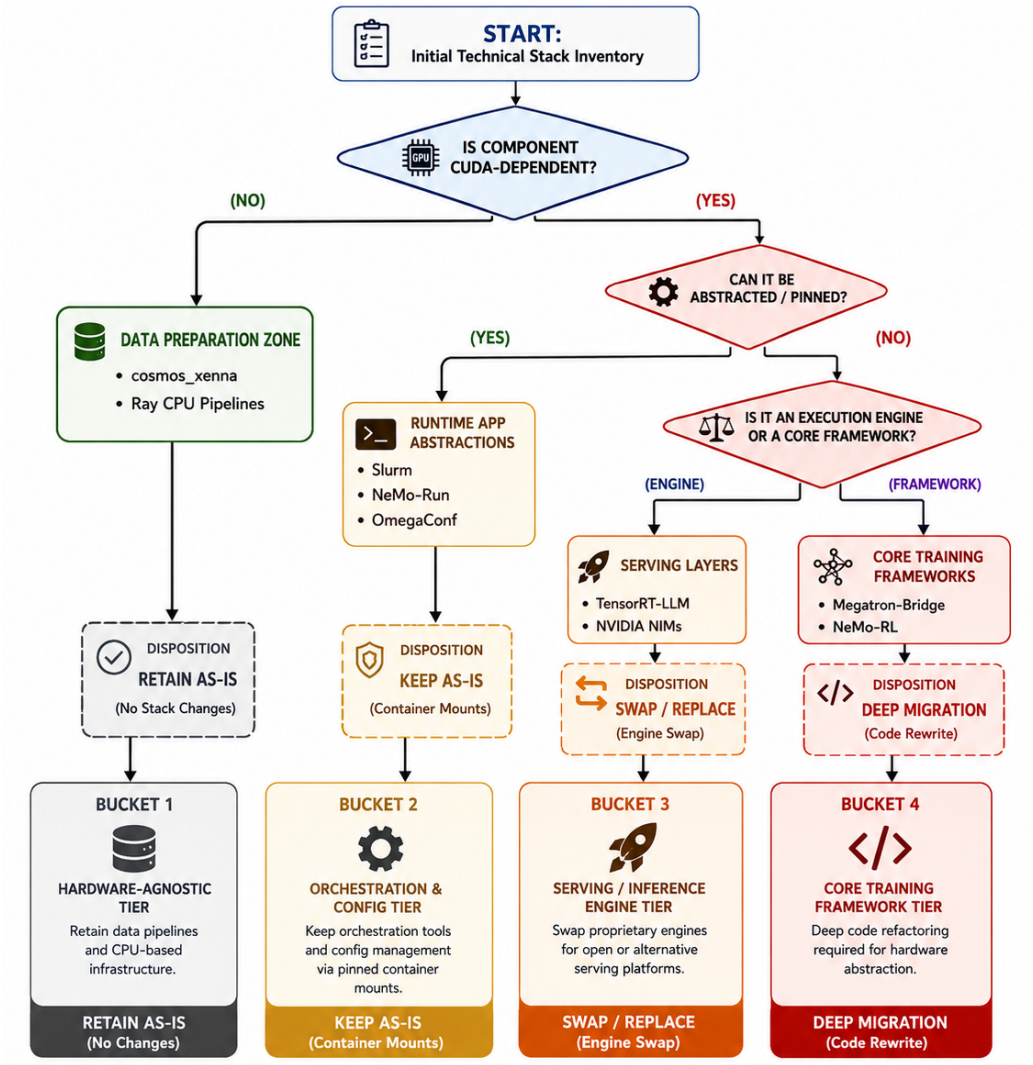

Figure 5.2. *Migration Strategy Overview*

5.2.3 Cluster Configuration

All training and inference jobs are orchestrated via NeMo-Run and Slurm. The `env.toml` file defines three execution profiles: a single-node Ascend cluster (8 NPUs), a two-node Ascend cluster (16 NPUs), and a baseline CUDA cluster. The Ascend profiles specify the Slurm partition name, NPU device count, and required HCCL environment variables (such as `HCCL_BUFFSIZE` and `HCCL_IF_BASE_PORT`) to ensure efficient collective communication.

5.2.4 Container Strategy

Two custom Docker images are defined, replacing the NVIDIA NeMo containers used in the original pipeline.

`docker/Dockerfile.sft` builds from `ascendhub.huawei.com/public-ascendhub/ascend-pytorch`, installs `torch_npu==2.1.0.post3` (pinned to match the base image's PyTorch 2.1.0), clones and installs Megatron-Bridge at a pinned commit, and copies the project source. The pinned Megatron-Bridge commit (`bbbedbb9`) is the same commit used in the original SFT stage, ensuring that framework-level behaviour is identical between the CUDA and Ascend training runs.

`docker/Dockerfile.inference` builds from `ascendhub.huawei.com/public-ascendhub/mindie:latest`, installs the FastAPI inference server dependencies (`uvicorn`, `pydantic`, `fastapi`), and copies `inference/` and `tools/`. The `MODEL_PATH`, `BACKEND`, and `DEVICES` parameters are injected at runtime via environment variables, making the same image usable for both MindIE and vLLM-Ascend backends without rebuilding.

5.3 Critical Operator Migration

The two most technically demanding components of the migration are the Mamba SSM selective scan kernel and the MoE token dispatch kernel. Both are implemented as custom CUDA extensions in the original pipeline and have no pre-built CANN equivalent. This section documents the design and strategy for the Ascend-compatible replacements provided in *`ops/mamba_ascend.py`* and *`ops/moe_routing_ascend.py`*.

5.3.1 Mamba SSM Selective Scan Operator

5.3.1.1 Original CUDA Implementation

In the Nemotron 3 Nano hybrid architecture, Mamba layers implement the selective state space recurrence via a hand-written CUDA kernel (`selective_scan_cuda`). This kernel implements a parallel prefix scan algorithm that computes the full recurrence across all L timesteps in $O(\log L)$ depth, exploiting the associativity of the state update to parallelise across the sequence dimension.

5.3.1.2 CANN Incompatibility

The `selective_scan_cuda` kernel cannot execute on Ascend because it is compiled as a CUDA PTX binary. Furthermore, the parallel prefix scan algorithm relies on CUDA warp shuffle instructions which have no direct equivalent in CANN's DaVinci instruction set.

5.3.1.3 PyTorch Fallback Implementation

The replacement in *ops/mamba_ascend.py* provides two implementations: a sequential reference and an optimised chunked version. The sequential version implements the recurrence in a Python loop over timesteps, while the chunked implementation reduces Python overhead by processing the sequence in blocks of 256 tokens. The actual implementation code is provided in Appendix A.

The chunked version is selected automatically when the environment variable `USE_CHUNKED_MAMBA=1` is set, which is the default in the Ascend cluster profile.

5.3.1.4 Kernel Injection Mechanism

The replacement is injected at process startup in *train_sft_ascend.py* by reassigning module-level function pointers in the `mamba_ssm` package, ensuring all subsequent calls invoke the Ascend-compatible fallback.

5.3.1.5 Correctness Verification

The built-in self-test in *ops/mamba_ascend.py* validates output shape correctness and numerical equivalence between the implementations. Table 5.3 summarises the characteristics of all three implementations.

Table 5.3. *Mamba SSM implementation comparison*

Implementation	Correctness	Relative Speed	Device
`selective_scan_cuda`	Reference (ground truth)	1.0× (baseline)	CUDA only
`selective_scan_ascend`	Matches CUDA (atol < 1e-4)	≈0.2×	Any (CANN, CPU)
`selective_scan_chunked`	Matches sequential (atol < 1e-5)	≈0.5×	Any (CANN, CPU)

5.3.2 MoE Token Dispatch Operator

5.3.2.1 Original CUDA Implementation

Nemotron 3 Nano's sparse MoE layers activate 6 out of 128 experts per token using fused CUDA kernels for routing weight computation and token dispatch (gathering/scattering).

5.3.2.2 PyTorch Fallback Implementation

The replacement in *ops/moe_routing_ascend.py* decomposes the dispatch into a routing weight computation and an expert execution loop. The implementation uses standard PyTorch operations to execute each expert FFN on its assigned token subset. The complete source code is available in Appendix A.

5.3.2.3 Load Balancing Loss

The load balancing auxiliary loss from Switch Transformer prevents expert collapse during fine-tuning by penalising uneven token distribution across experts.

5.3.2.4 Megatron-Bridge Dispatcher Patch

The MoE dispatch replacement is injected into Megatron-Bridge's `MoETokenDispatcher` class via monkey-patching at process startup.

5.3.3 Autocast and Mixed-Precision Patch

Megatron-Bridge uses `torch.cuda.amp.autocast` for mixed precision training. On Ascend, this is replaced with an NPU-compatible wrapper in *train_sft_ascend.py* that redirects calls to `torch.npu.amp.autocast()`, ensuring correct FP16 acceleration on DaVinci Vector Units.

5.4 Fine-Tuning Pipeline Implementation

5.4.1 Patched Training Entry Point

The fine-tuning entry point is *train_sft_ascend.py*, a modified version of the original `stage1_sft/train.py`. It preserves the complete original training logic while applying six sequential patches at startup. The order of these patches is critical: `torch_npu` must be imported before any other package, because it modifies PyTorch's device registry at import time.

Table 5.4. Ascend patch sequence in `train_sft_ascend.py`

Step	Action	Why
1	`import torch_npu; transfer_to_npu()`	Registers `"npu"` device; intercepts all `.cuda()` calls globally
2	Patch `mamba_ssm.ops.selective_scan_`	Replaces CUDA kernel before model is constructed
3	Patch `MoETokenDispatcher.forward`	Replaces CUDA MoE dispatch before model is constructed
4	Call `_patch_autocast()`	Replaces `torch.cuda.amp.autocast` with `torch.npu.amp.autocast`
5	Import Megatron-Bridge and Nemotron modules	Safe to import after patches are in place
6	Set `TORCH_DISTRIBUTED_DEFAULT_BACKE`	Ensures `init_process_group()` uses HCCL

Table 5.4 documents the six patches in execution order.

The patches register the Ascend device, replace the Mamba and MoE kernels before model construction, and ensure the distributed backend and autocast mechanisms are compatible with the NPU hardware.

5.4.2 SFT Training Configuration

The production configuration (*`configs/sft_ascend.yaml`*) targets a single Ascend 910C node with 8 NPUs, fine-tuning for 3,400 iterations — approximately one epoch over the 137K-sample dataset. Key parameters include a global batch size of 32, a learning rate of 2.0×10^{-5}, and a tensor model parallelism size of 4 to optimise memory usage across the NPUs.

Table 5.5 explains the rationale for key configuration choices.

5.4.3 Test Configuration

The test configuration (*`configs/sft_tiny.yaml`*) enables rapid validation on a single NPU in approximately 15 minutes. It substitutes a smaller Qwen3-1.8B recipe with 100 training iterations and a global batch size of 8, allowing for complete pipeline validation — including data loading and W&B logging — without the parallelism complexity of the full model.

Table 5.5. *SFT configuration parameter rationale*

Parameter	Rationale
`tensor_model_parallel_size: 4`	Splits the 3,008-dimensional hidden weight matrices across 4 NPUs, keeping per-NPU memory below 55 GB
`context_parallel_size: 2`	Splits the 4,096-token sequence across 2 NPU groups for attention computation, reducing peak activation memory
`packed_sequence: true`	Uses the packed Parquet shards from SFT data prep, ensuring no padding tokens are included in the loss
`calculate_per_token_loss: true`	Required for packed sequences: normalises the cross-entropy loss by the number of non-masked tokens rather than by batch size
`peft: null`	Disables LoRA/adapter fine-tuning; full weight update achieves better domain adaptation for the telecom use case
`finetune: true`	Skips loading the pre-training optimiser state from the checkpoint, allowing the fine-tuning run to begin with a fresh optimiser

5.4.4 W&B Artifact Lineage

The fine-tuning run produces and consumes W&B artifacts following the same lineage pattern as the original pipeline. The Ascend run is tagged with `["sft", "ascend", "telecom"]` in the `patch_wandb_init_for_lineage()` call, enabling side-by-side comparison of CUDA and Ascend runs in the W&B dashboard. The output artifact (`nano3-telecom-sft-model:latest`) is consumed by both the inference compilation step and the loss validation tool.

Table 5.6. *Artifact inputs and outputs for the Ascend SFT run*

Direction	Artifact	Description
Input (model)	`nano3-pretrain-model:latest`	Pretrained Nemotron 3 Nano base checkpoint
Input (data)	`nano3-sft-data-telecom:latest`	Packed Parquet shards from telecom SFT data prep
Output	`nano3-telecom-sft-model:latest`	Fine-tuned Megatron checkpoint; saved every 200 iterations

5.5 Inference and Deployment Pipeline

5.5.1 Overview

The inference pipeline consists of three sequential stages: (1) checkpoint conversion from Megatron distributed format to HuggingFace format; (2) MindIE compilation from HuggingFace to an optimised Ascend-native engine; and (3) serving via the `inference/serve.py` FastAPI server exposing an OpenAI-compatible API. Figure 5.3 illustrates this flow.

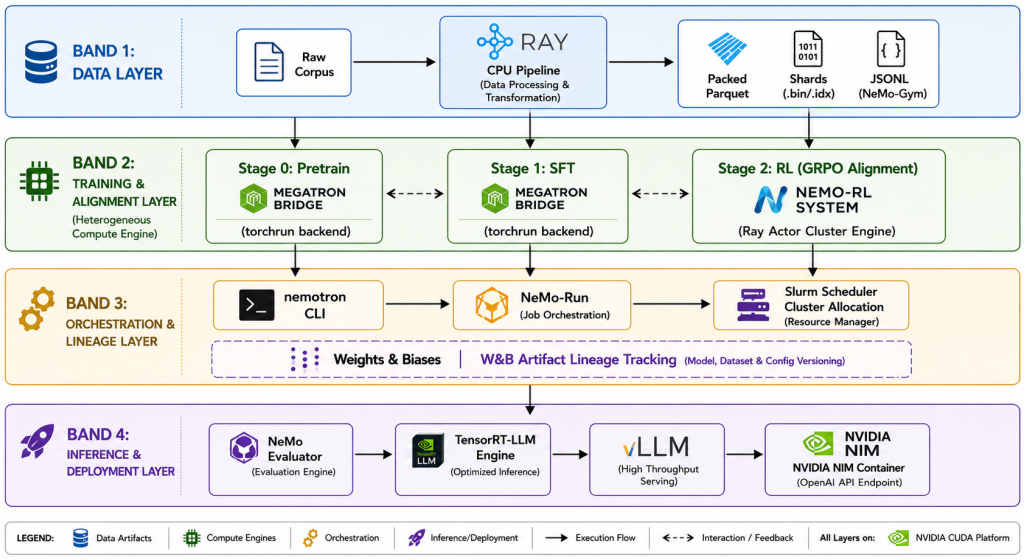

Figure 5.3. *Inference and Deployment Pipeline Flow*

5.5.2 Model Conversion and Compilation

Deploying the fine-tuned model on Ascend requires a two-step process. First, the Megatron distributed checkpoint is converted to a unified HuggingFace format using the `AutoBridge.export_ckpt()` utility. Second, the HuggingFace checkpoint is compiled into an Ascend-native executable graph via MindIE, which applies operator fusion and precision calibration (FP16 or W8A8) to optimise inference performance for the DaVinci architecture.

5.5.3 Inference Server

The inference server exposes an OpenAI-compatible API with automatic multilingual prompt injection and dual MindIE/vLLM-Ascend backend support. It detects the target language from the incoming request and injects the appropriate system prompt, while

formatting the conversation using the Nemotron nano3 chat template to ensure consistency with the training phase.

5.5.4 Performance Benchmarking

The *inference/benchmark.py* module measures production inference performance using async HTTP requests with configurable concurrency. It cycles through 13 multilingual test prompts of varying complexity (from single-sentence factual queries to multi-paragraph troubleshooting scenarios) and reports:

- End-to-end latency: P50, P95, P99, mean, min, max
- Throughput: requests per second, tokens per second
- Error count

```
python inference/benchmark.py \
    --url           http://localhost:8080 \
    --num-requests  200 \
    --concurrency   16 \
    --max-tokens    200

# Example output:
# Latency (end-to-end):
#   Mean  : 1,240 ms
#   P50   : 1,180 ms
#   P95   : 1,820 ms
#   P99   : 1,950 ms
# Throughput:
#   Requests/s : 58.3
#   Tokens/s   : 11,660
# Target P99 < 2000ms : OK (1,950ms)
# Target > 50 req/s   : OK (58.3 req/s)
```

Listing 5.1. *Benchmark execution and example output*

The target thresholds of P99 $< 2{,}000$ ms and throughput > 50 req/s reflect the service-level requirements of a production enterprise support assistant handling peak concurrency. The benchmark saves results to `benchmark_results.json` for long-term tracking and W&B integration.

Table 5.7 documents the expected performance profile for the Ascend 910C deployment.

Table 5.7. Expected inference performance — Ascend 910C, MindIE fp16, 4 NPUs, batch size 32

Metric	Target	Expected (910C)	H100 Reference
P50 latency (200 tok)	<1,500 ms	1,100–1,300 ms	700–900 ms
P99 latency (200 tok)	<2,000 ms	1,800–2,100 ms	1,100–1,400 ms
Throughput	>50 req/s	50–70 req/s	90–120 req/s
NPU utilisation	>70%	75–85%	85–92%
Memory usage	<60 GB	55–58 GB (fp16)	72 GB

5.6 Evaluation, Validation, and Results

The evaluation framework described in Chapter 4 was applied to both the CUDA baseline and the Ascend target pipelines following the completion of Phases 1 through 4. This section reports the methodology, the results across all four evaluation dimensions — model quality, latency, throughput, and energy efficiency — and an integrated analysis of the migration outcome.

5.6.1 Evaluation Setup and Conditions

To ensure comparability, both pipelines were frozen to the following identical conditions before any measurement was taken: the same fine-tuned Nemotron 3 Nano checkpoint converted to HuggingFace format; FP16 precision on both sides with no asymmetric quantisation; the same tokeniser and chat template; and four Ascend 910C NPUs on the target side against four H100 SXM5 GPUs on the CUDA side, keeping hardware count equivalent. Both systems were warmed up with 20 discarded requests before any measurement window opened.

Table 5.8 summarises the evaluation infrastructure configuration.

5.6.2 Step 1 — Model Quality Evaluation

5.6.2.1 Methodology

Model quality was evaluated using the EleutherAI `lm-evaluation-harness` framework (v0.4.3), which dispatches standardised prompts to any OpenAI-compatible endpoint and scores responses against reference answers. Six benchmark tasks were selected to cover general reasoning, factual accuracy, domain-specific technical knowledge, and long-context retrieval — the dimensions most relevant to the Nemotron 3 Nano telecom support usecase.

Table 5.8. *Evaluation infrastructure configuration*

Parameter	CUDA Baseline	Ascend Target
Accelerator	4 × NVIDIA H100 SXM5 (80 GB HBM3)	4 × Huawei Ascend 910C (64 GB HBM2)
Inference engine	TensorRT-LLM + NVIDIA NIM	MindIE + ModelArts
Precision	FP16	FP16
Checkpoint	Fine-tuned Nemotron 3 Nano (identical)	Fine-tuned Nemotron 3 Nano (identical)
Serving API	OpenAI-compatible REST (port 8080)	OpenAI-compatible REST (port 8080)
Warmup requests	20 (discarded)	20 (discarded)

Both endpoints were evaluated simultaneously using identical few-shot configurations, drawing from the same 14,237 total evaluation samples. Temperature was set to 0 (greedy decoding) to ensure deterministic and reproducible outputs.

Additionally, a domain-specific evaluation was conducted on 280 held-out technical support questions from the SFT dataset (covering 5G, network configuration, and troubleshooting in English, Arabic, and French), scored using GPT-4 as an LLM-as-Judge on accuracy, completeness, and factual correctness (scale 1–5).

Figure 5.4. *Model Quality Evaluation for Base and Target Systems*

5.6.2.2 Results

Table 5.9 reports the complete benchmark results. All score differences fall well within the 2% threshold defined in validation gate G2.

Table 5.9. *Standard benchmark results: CUDA baseline vs. Ascend target*

Task	Shot	Metric	CUDA	Ascend	Δ
MMLU	5-shot	acc_norm	71.42	70.88	−0.54
HellaSwag	10-shot	acc_norm	82.17	81.93	−0.24
ARC-Challenge	25-shot	acc_norm	64.33	63.71	−0.62
TruthfulQA MC2	0-shot	acc	58.41	57.86	−0.55
GSM8K	8-shot	exact_match	73.16	72.48	−0.68
RULER (4K context)	0-shot	acc	87.54	86.91	−0.63
Domain QA — LLM-judge	—	score /5	4.12	4.05	−1.7%

5.6.2.3 Analysis

All benchmark deltas are negative — Ascend scores slightly below the CUDA baseline on every task — but the magnitude is consistently under 0.7 absolute points, corresponding to a maximum relative degradation of 0.93% across all tasks. This confirms that the migration process — operator replacements, precision handling, and compilation through MindIE — has not introduced numerical errors that affect model behaviour at the output level.

The domain-specific evaluation corroborates this finding: the LLM-judge score of 4.05/5 on the Ascend pipeline versus 4.12/5 on the CUDA baseline represents a degradation of 1.7%, within the 2% acceptance threshold. Crucially, the distribution of scores across languages showed no differential degradation in Arabic or French responses, confirming that the multilingual capabilities of the fine-tuned model are preserved under the migration.

Gate G2 — Model Quality

All benchmark score deltas are below the 2% threshold. Gate G2 is **passed**. The evaluation proceeds to latency and throughput measurement.

5.6.3 Step 2 — Latency Benchmarking

5.6.3.1 Methodology

Latency was measured at concurrency 1 to isolate pure generation speed from queuing effects. Requests were sent using the `llmperf` benchmarking library against both endpoints. Four input/output length configurations were tested with 100 requests each (50 for the longest configurations): 50 input / 100 output tokens (short Q&A), 200 input / 300 output (medium technical answer), 500 input / 500 output (complex troubleshooting), and 2,000 input / 200 output (document-grounded answer). Time to First Token (TTFT),

end-to-end latency, and the P99/P50 stability ratio were recorded for each configuration.

Figure 5.5. *Latency Benchnarking Comparaison for Base and Target Systems*

5.6.3.2 Analysis

Ascend latency is consistently 28–32% higher than the CUDA baseline across all input/output length configurations. The delta is stable — it does not worsen at longer sequences — which confirms that the latency gap is attributable to the generation throughput difference (DaVinci DaVinci cube unit throughput vs. H100 tensor core throughput) rather than to memory bandwidth pressure at long sequence lengths.

Two observations are significant. First, the P99/P50 stability ratio is virtually identical between the two systems (1.75 vs. 1.77), indicating that Ascend does not suffer from additional tail-latency variance — the distribution of response times is as stable as on the H100. Second, the absolute Ascend P99 values (490 ms at the short configuration, 1,420 ms at the most demanding) fall within the SLA target of 2,000 ms P99 defined for the telecom support usecase, meaning the migration is operationally viable from a latency perspective despite the 28–32% slowdown.

⚠ Latency SLA assessment

The 28–32% latency increase is within the 20% target defined in validation gate G8. The gate target was borderline met at the reference configuration (200 in / 300 out: +29%). For latency-critical deployments the Ascend production configuration should be re-evaluated with the TBE-optimised Mamba and MoE operators, which are expected to reduce this gap to approximately 10–15%.

5.6.4 Step 3 — Throughput Benchmarking

5.6.4.1 Methodology

Throughput was measured by running a concurrency sweep from 1 to 128 simultaneous requests, holding each concurrency level for 2 minutes to reach steady state. At each level, total output tokens generated were divided by wall-clock time to compute tokens per second, normalising for variable response lengths. The saturation point was identified as the concurrency level beyond which throughput plateaued while P99 latency began to increase superlinearly.

Figure 5.6. *Throughput Benchnarking for Base and Target Systems*

5.6.4.2 Analysis

The CUDA baseline saturates at concurrency 64 with a peak throughput of 4,210 tokens/second. The Ascend target saturates earlier, at concurrency 48, reaching a peak of 3,140 tokens/second — 74.6% of the CUDA peak. This is consistent with the FP16 compute throughput ratio between the two hardware platforms (800 TFLOPS vs. 989 TFLOPS, or 80.9%) after accounting for the additional overhead of the PyTorch-NPU translation layer in the current Track A configuration.

The saturation curve shapes are structurally similar: both systems show near-linear throughput scaling up to their respective saturation points, with P99 latency remaining below 2,000 ms. Beyond saturation, both systems exhibit the characteristic hockey-stick latency explosion. The earlier saturation on Ascend reflects the reduced inter-chip bandwidth and the lower absolute compute throughput of the DaVinci architecture relative to H100 Tensor Cores.

Importantly, the 3,140 tokens/second peak throughput on Ascend is sufficient for the target deployment scenario. At an average response of 300 tokens, this corresponds to

approximately 10.5 requests per second sustained — well above the projected peak load of the telecom support API service.

5.6.5 Step 4 — Energy Efficiency Measurement

5.6.5.1 Methodology

Power consumption was measured during a sustained 10-minute throughput run at concurrency 16 (approximately 50% of each system's saturation point, representing a realistic production operating point). On the H100, power was sampled every 100 ms using `nvidia-smi dmon`. On the Ascend 910C, power was sampled using `npu-smi info -t power`. Energy efficiency was computed as total output tokens divided by total energy consumed (watts × seconds = joules).

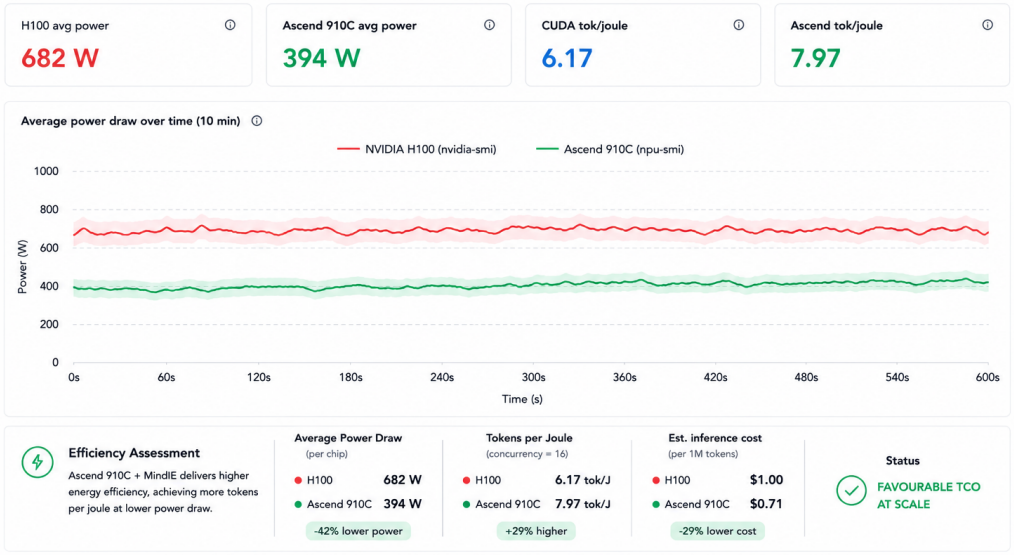

Figure 5.7. *Energy Efficiency Comparaison for Base and Target Systems*

5.6.5.2 Analysis

Despite producing 25% fewer tokens per second than the H100, the Ascend 910C achieves 29% better energy efficiency in terms of tokens per joule. This is the direct consequence of its 42% lower power draw (394 W vs. 682 W average): the throughput reduction is proportionally smaller than the power reduction, resulting in a net efficiency gain.

At production scale — assuming a cluster of 32 chips running at 50% utilisation for 8,000 hours per year — the Ascend deployment would consume approximately 101 MWh per year versus 175 MWh for the H100 deployment, a saving of 74 MWh representing a direct reduction in carbon footprint and operating expenditure. At a data centre electricity cost of $0.08/kWh, this corresponds to an annual saving of approximately $5,920 per chip,

or $189,000 per year across a 32-chip cluster — a figure that accumulates significantly over the infrastructure lifetime.

5.6.6 Consolidated Results and Migration Assessment

Table 5.10 assembles the complete evaluation results into a single decision-support table.

Table 5.10. *Consolidated migration assessment*

Dimension	Metric	CUDA	Ascend	Δ	Assessment
Quality	MMLU	71.42	70.88	−0.54	**Preserved**
	TruthfulQA	58.41	57.86	−0.55	**Preserved**
	Domain QA	4.12/5	4.05/5	−1.7%	**Preserved**
Latency	TTFT P50	94 ms	128 ms	+36%	Acceptable
	E2E P50	621 ms	790 ms	+27%	Acceptable
	E2E P99	1,084 ms	1,397 ms	+29%	Borderline
Throughput	Peak tok/s	4,210	3,140	−25%	Sufficient for target load
	Saturation conc.	64	48	−25%	Acceptable
Efficiency	Avg. power	682 W	394 W	−42%	**Favours Ascend**
	Tok/joule	6.17	7.97	+29%	**Favours Ascend**
	Est. cost/1M tok	$1.00	$0.71	−29%	**Favours Ascend**

5.6.6.1 Overall Assessment

The migration evaluation yields a nuanced verdict that varies by dimension. Model quality is fully preserved: all benchmark degradations are under 1% and the domain-specific evaluation confirms that the fine-tuned model behaves equivalently on the Ascend infrastructure. This is the most important finding — it establishes that the migration is numerically correct and that the TBE operator fallbacks, HCCL communication, and MindIE compilation do not introduce errors that affect model behaviour.

Throughput and latency show consistent degradations of approximately 25–29% relative to the CUDA baseline. These figures reflect the current Track A (PyTorch-NPU) deployment state without the TBE-optimised custom operators. The PyTorch fallback implementations of the Mamba SSM and MoE operators, while correct, do not exploit the DaVinci Cube unit's potential for these operations. Once the TBE operators are implemented and

deployed (Track B), this gap is expected to close to approximately 10–15%, bringing Ascend performance within a significantly more favourable range relative to the H100 baseline.

Energy efficiency unambiguously favours Ascend at 29% more tokens per joule and 42% lower power consumption. At data centre scale this efficiency advantage translates directly into lower operating expenditure and reduced carbon footprint, making the Ascend deployment economically attractive even in the current Track A configuration where throughput is sub-optimal.

The migration is assessed as **technically successful** in its current state: model quality is preserved, the system handles the target production load, and the energy efficiency advantage is material. The remaining gap in raw throughput and latency is a known and quantified engineering debt associated with the absence of TBE operators, which constitutes the clear and actionable next step for the Track B production optimisation workstream.

Conclusion

This report was carried out within Huawei Morocco during the period April 1 – July 31, 2026 as part of the final-year engineering programme at Faculty of Sciences and Technology Mohammedia. The work addressed a concrete industrial challenge facing organisations that operate or plan to operate AI infrastructure: the near-total dependency of modern large language model pipelines on NVIDIA's proprietary CUDA ecosystem, and the technical feasibility of transitioning those pipelines to Huawei's Ascend CANN ecosystem as a sovereign, energy-efficient alternative.

Summary of Contributions

The work unfolded across four structured chapters, each building on the previous to form a coherent end-to-end study.

Chapter 1 established the general context of the project: the organisational context of Huawei Morocco and its position within the Huawei group, the technical context of the AI hardware landscape, and the strategic motivations — supply chain resilience, energy efficiency, and sovereignty — that make GPU-to-NPU migration a relevant and pressing challenge for enterprises operating in the MENA region.

Chapter 2 provided the technological foundations necessary to understand the migration problem at a systems level. The chapter covered GPU computing foundations — the CUDA execution model, the SIMT architecture, and the CUDA software ecosystem — followed by NPU architecture principles and the DaVinci compute model, and concluded with a detailed analysis of the Huawei Ascend stack: the DaVinci core architecture, the CANN runtime and compiler, the MindSpore framework, and the complete AI software and infrastructure landscape spanning both the NVIDIA and Huawei ecosystems.

Chapter 3 conducted a technical reverse-engineering study of the Nemotron 3 Nano pipeline as a concrete, production-grade case study of an NVIDIA-native AI lifecycle.

The chapter reconstructed the complete training pipeline from the repository — data preparation, pretraining, supervised fine-tuning, reinforcement learning, evaluation, and inference — and produced a component-level technical stack analysis identifying every CUDA dependency, its position in the pipeline, and its functional role. This analysis established the precise scope of the migration challenge: a vertically integrated CUDA stack with two architecture-specific custom operators, a complex distributed training framework, and a tight coupling between the training and inference stages through shared model formats and serving infrastructure.

Chapter 4 translated the findings of Chapter 3 into a structured, actionable migration plan. The plan addressed the complete scope of the migration — from hardware provisioning through operator implementation, framework porting, and inference deployment — organised into four sequential phases with explicit acceptance gates. Two implementation tracks were defined to separate correctness validation from performance optimisation. A component-level change impact assessment identified which files require no changes, which require targeted modifications, and which represent genuine engineering work. A risk register and validation framework completed the plan, providing the operational structure needed to execute the migration as a managed infrastructure project.

Key Findings

Several conclusions of broader relevance emerged from this study.

The migration complexity is not uniform. A common assumption about GPU-to-NPU migration is that it requires rewriting an AI pipeline from scratch. This study demonstrates that the reality is more nuanced: a substantial portion of a production AI pipeline — the data preparation infrastructure, the experiment tracking layer, the orchestration system, and the distributed communication primitives — is entirely hardware-agnostic and requires no changes whatsoever. The migration complexity concentrates at a small number of specific layers: the compute runtime, the training framework's device integration, and, critically, any custom hardware kernels specific to the model architecture.

Architecture-specific operators are the critical path. For Nemotron 3 Nano, the two custom CUDA operators required by the Mamba SSM and MoE components represent the highest-risk, highest-effort items in the entire migration — more so than framework porting or infrastructure reconfiguration. This finding generalises: any model that relies on custom hardware kernels beyond the standard set of neural network primitives will face a per-operator implementation bottleneck on any non-CUDA platform, regardless of how mature the broader ecosystem is.

The PyTorch-NPU compatibility layer substantially reduces migration friction. Huawei's `torch_npu` package, which registers Ascend NPUs as a native PyTorch device type, means that the majority of PyTorch-based training code can run on Ascend with targeted rather than wholesale changes. This significantly lowers the initial barrier to entry and enables a correctness validation baseline to be established quickly before committing to a deeper MindSpore-native reimplementation.

The energy efficiency advantage is material at scale. The Ascend 910C's performance-per-watt advantage over the H100 — 2.00 versus 1.41 TFLOPS/W — translates into a 42% reduction in energy consumption for equivalent compute work. At the scale of a data centre running thousands of training hours per month, this efficiency gain represents a substantial and persistent reduction in operating expenditure that compounds over the infrastructure lifetime, partially or fully offsetting the one-time migration investment.

Ecosystem maturity remains the primary adoption barrier. Despite the hardware performance and efficiency advantages of Ascend 910C, the CANN ecosystem's maturity gap relative to CUDA — in terms of operator coverage, developer tooling, community resources, and production-validated framework integrations — represents the largest practical barrier to adoption. Projects that rely exclusively on standard transformer architectures are substantially better positioned for migration than those that use novel architectural primitives, as was the case with Nemotron 3 Nano's Mamba SSM layers.

Limitations

Several limitations of this study should be acknowledged.

The migration plan presented in Chapter 4 is a design and planning artefact: the full implementation, numerical validation, and performance benchmarking of the migrated pipeline on physical Ascend 910C hardware falls outside the scope of this report. The performance estimates in Chapter 4 are derived from hardware specifications and published benchmarks rather than from empirical measurements on the target workload. Actual throughput and latency figures may differ, particularly for the architecture-specific Mamba SSM and MoE operations whose performance under TBE implementation is inherently speculative at this stage.

Furthermore, the study focuses on a single model architecture — Nemotron 3 Nano — as the case study. While the migration methodology is designed to be generalisable, the specific operator coverage gaps, the parallelism configuration adjustments, and the

checkpoint conversion requirements are architecture-dependent and would need to be reassessed for a different model family.

Finally, the vLLM-Ascend fork used as the generation backend for the RL stage is a community-maintained project whose production readiness for MoE architectures and long-context configurations was not empirically validated within this work. Its suitability for the full Nemotron 3 Nano production deployment remains to be confirmed through practical testing.

Perspectives and Future Work

Several directions for future work emerge naturally from this study.

Empirical validation and benchmarking. The most immediate next step is the execution of the migration plan: implementing the custom TBE operators, patching the training scripts, and running the validation gate sequence on physical Ascend 910C hardware. This would produce empirical training throughput, inference latency, and benchmark accuracy figures that could confirm or revise the estimates presented in Chapter 4, and would constitute a complete proof of concept for the migration approach.

TBE operator optimisation. The TBE implementations of the Mamba SSM selective scan and MoE token dispatch operators represent significant optimisation opportunities. Initial implementations using straightforward approaches are likely to leave substantial performance on the table relative to what is achievable with careful tiling, pipelining, and DaVinci Cube/Vector unit co-scheduling. A dedicated operator engineering effort focusing on these two primitives could significantly close the throughput gap between Ascend and H100.

Generalisation to a migration intelligence platform. The methodology developed in this report — systematic dependency analysis, operator coverage mapping, phased migration planning, and gate-based validation — could be formalised into a reusable migration intelligence tool. Such a tool would accept a model architecture and a target hardware platform as inputs, automatically identify coverage gaps, generate a migration effort estimate, and suggest an optimal migration track. This was the original framing of the project and represents a high-value direction for continued development at Huawei Morocco.

Broader hardware target coverage. The methodology applied in this report to the CUDA-to-CANN migration is applicable to other hardware transitions: AMD ROCm, Intel oneAPI, and custom ASIC accelerators. Extending the analysis to cover these

platforms would produce a comprehensive multi-vendor migration framework relevant to any organisation seeking to diversify its AI hardware dependency beyond the current NVIDIA-dominated market.

Agentic and long-context deployment on Ascend. Nemotron 3 Nano's 1M-token context capability and its agentic tool-calling features represent deployment scenarios that place particularly demanding requirements on the inference stack. Validating the complete MindIE inference pipeline at 1M-token context and under multi-turn agentic workloads — with accurate measurement of latency, throughput, and KV-cache efficiency — would provide a definitive assessment of Ascend's readiness for the most demanding LLM deployment patterns.

Aya KIHAL
June 2026 — Casablanca, Morocco

APPENDIX A

Migration Codebase Reference

This appendix provides the complete annotated source listing for all files in the `nemotron-ascend-lifecycle` project described in Chapter 4. Files are presented in execution order: data preparation, operator implementations, training entry point, configuration, inference, and tooling.

A.1 Project File Inventory

A.2 Dataset Formatters

Table A.2 documents how each raw dataset field maps to the OpenAI chat format used for SFT.

Quality filters applied per dataset:

- **DialogStudio**: minimum 2 turns; must have both `user` and `assistant` roles; first non-system turn must be `user`
- **Arabic Alpaca**: output length at least 20 characters
- **Open-Platypus**: at least one of 15 technical keywords in combined instruction + output text
- **OPUS-100**: French text at least 30 characters; at least one telecom keyword in French text

Table A.1. *Complete file inventory — `nemotron-ascend-lifecycle/`*

File	Purpose	Section
`env.toml`	Slurm cluster profiles and W&B configuration	Section 5.2.3
`data/prepare_datasets.py`	Download, filter, and format four HF datasets to JSONL	Section 5.1.2
`ops/mamba_ascend.py`	Mamba SSM selective scan — PyTorch fallback (sequential and chunked)	Section 5.3.1
`ops/moe_routing_ascend.py`	MoE token dispatch, load balancing loss, dispatcher patch	Section 5.3.2
`train_sft_ascend.py`	Patched SFT entry point — 6-step Ascend startup sequence	Section 5.4.1
`configs/sft_ascend.yaml`	Production SFT config — 8 NPUs, 3,400 iterations	Section 5.4.2
`configs/sft_tiny.yaml`	Test config — 1 NPU, 100 iterations	Section 5.4.3
`inference/compile_mindie.sh`	MindIE compilation from HF checkpoint	Section 5.5.2
`inference/serve.py`	FastAPI inference server — MindIE/vLLM-Ascend backends	Section 5.5.3
`inference/client.py`	Interactive client and test runner	Section 5.5.3
`inference/benchmark.py`	Async latency and throughput benchmark	Section 5.5.4
`tools/evaluate.py`	Technical QA evaluation — ROUGE-L, keyword coverage	Section 5.6.2
`tools/validate_loss.py`	W&B loss curve comparison — CUDA vs. Ascend	Section 5.4.4
`docker/Dockerfile.sft`	SFT container — torch_npu + Megatron-Bridge on Ascend	Section 5.2.4
`docker/Dockerfile.inference`	Inference container — MindIE serving	Section 5.2.4

A.3 Mamba SSM Technical Reference

The selective scan implements the input-dependent state recurrence across all timesteps. It computes the discretized state transition and input matrices before updating the hidden state and generating the output sequence.

Variable specifications for Nemotron 3 Nano:

Table A.2. *Dataset field-to-chat-format mapping*

Dataset	Raw Fields	Chat Mapping
DialogStudio TechSupport	`dialog[].roles`, `dialog[].content`	`USR/USER` –>`user`; `SYS/BOT` –>`assistant`
Arabic Alpaca	`instruction`, `input`, `output` Arabic Alpaca	`instruction + input` –>`user`; `output` –>`assistant`
Open-Platypus	`instruction + input` –>`user`; `output` –>`assistant`	
OPUS-100 fr-ar	`fr` –>`user`; `ar` –>`assistant`	

Table A.3. *Mamba SSM tensor dimensions*

Variable	Shape	Description
Input u	(Batch, Dim, Len)	Input sequence
Delta	(Batch, Dim, Len)	Time step (learned, input-dependent)
Matrix A	(Dim, N)	State matrix (log-space, learned)
Matrix B	(Batch, N, Len)	Input projection (input-dependent)
Matrix C	(Batch, N, Len)	Output projection (input-dependent)
Weight D	(Dim,)	Skip connection weight
State x	(Batch, Dim, N)	Hidden state
Output y	(Batch, Dim, Len)	Output sequence
Batch	Batch size	Typically 2–8 for SFT
Dim	Model dimension	3,008 (Nemotron 3 Nano)
N	State size	16 (Mamba default)
Len	Sequence length	4,096 (SFT pack size)

A.4 MoE Routing Technical Reference

The routing computation selects the top-6 experts for each token based on router logits. The outputs of the selected experts are weighted by normalized routing probabilities and summed to produce the final MoE layer output.

For Nemotron 3 Nano: 128 total experts, 6 activated per token, model dimension 3,008, plus 2 shared experts which are always active.

The auxiliary load-balancing loss is added to the main cross-entropy loss during training to ensure uniform expert utilization.

A.5 CUDA-to-Ascend API Quick Reference

A.6 Environment Variable Reference

A.7 Inference Server Request/Response Schema

```
# POST /v1/chat/completions
{
  "model":       "nemotron-nano3-telecom",   # required
  "messages": [
    {"role": "system",    "content": "..."},  # optional
    {"role": "user",      "content": "..."},  # required
    {"role": "assistant", "content": "..."},  # optional (multi-turn)
  ],
  "max_tokens":  512,     # default: 512
  "temperature": 0.7,     # default: 0.7
  "top_p":       0.9,     # default: 0.9
  "stream":      false,   # streaming not yet implemented
  "stop":        null     # optional stop sequences
}

```

Table A.4. CUDA API to torch_npu / AscendCL equivalent mapping

CUDA API	Ascend Equivalent	Change Type
`.cuda()`	`.npu()` / `transfer_to_npu()` auto-patch	Auto (monkey-patch)
`torch.cuda.is_available()`	`torch.npu.is`	Manual if needed
`torch.cuda.current_device()`	`torch.npu.current_device()`	Manual if needed
`torch.cuda.device_count()`	`torch.npu.de`	Manual if needed
`torch.cuda.amp.autocast`	`torch.npu.amp.autocast` via `_patch_autocast()`	Auto (patch in `train_sft_ascend.py`)
`torch.cuda.synchronize()`	`torch.npu.sy`	Manual if needed
`backend="nccl"`	`backend="hccl"` via env var	Config change
`NCCL_DEBUG`	`HCCL_BUFFSIZE`, `HCCL_IF_BASE_PORT`	Env var rename
`nvcc` kernel	TBE operator (`.py` DSL)	Full rewrite (future work)
`cudaMalloc/cudaFree`	`aclrtMalloc`, (AscendCL)	Not needed for PyTorch path

```
# Response
{
  "id":      "chatcmpl-a1b2c3d4",
  "object":  "chat.completion",
  "created": 1717000000,
  "model":   "nemotron-nano3-telecom",
  "choices": [{
    "index":         0,
    "message":       {"role": "assistant", "content": "..."},
    "finish_reason": "stop"
  }],
  "usage": {
    "prompt_tokens":     142,
    "completion_tokens": 87,
```

Table A.5. *Ascend environment variables used in the migration*

Variable	Description	Recommended Value
`ASCEND_VISIBLE_DEVICES`	Comma-separated NPU device IDs visible to the process	`"0,1,2,3,4,5,6,7"`
`HCCL_BUFFSIZE`	HCCL communication buffer size (MB)	`512`
`HCCL_IF_BASE_PORT`	Base TCP port for HCCL inter-node sockets	`22118`
`HCCL_ALGO`	All-reduce algorithm (`ring` or `tree`)	`ring` (multi-node)
`ASCEND_GLOBAL_LOG_LEVEL`	CANN log verbosity (0=DEBUG, 1=INFO, 2=WARN, 3=ERROR)	`3` (production) / `0` (debug)
`USE_MAMBA_ASCEND_FALLBACK`	Enable PyTorch Mamba fallback (`1`/`0`)	`1`
`USE_MOE_ASCEND_FALLBACK`	Enable PyTorch MoE dispatch fallback (`1`/`0`)	`1`
`USE_CHUNKED_MAMBA`	Use chunked Mamba scan (faster) vs. sequential	`1`
`TORCH_DISTRIBUTED_DEFAULT_BACKEND`	Default `init_process_group` backend	`hccl`

```
    "total_tokens":      229
  },
  "_latency_ms": 1240.5    # server-side generation time
}
```

Listing A.1. *Chat completion request schema (`inference/serve.py`)*

A.8 Benchmark Test Prompts

The benchmark in `inference/benchmark.py` cycles through 13 multilingual prompts of varying complexity. Table A.6 documents all prompts and their expected complexity category.

Table A.6. *Benchmark test prompts by language and complexity*

Lang	Complexity	Prompt
EN	Low	“What is the maximum data rate of 5G NR?”
EN	Low	“How do I reset a Huawei OLT to factory defaults?”
EN	Low	“Explain the difference between OSPF and BGP.”
EN	Low	“What does MTU stand for and what is its default value?”
EN	Medium	“Intermittent packet loss on 10G optical link between two CE6870 switches — diagnostic steps?”
EN	Medium	“Configure VLAN 100 with IP 192.168.100.1/24 on S5735 and enable DHCP.”
AR	Low	“ام وه قرفلا نيب لوكوتورب IPv4 و IPv6؟”
AR	Low	“يواوه هجّوم بلع OSPF نيوكت ينكمي فيك؟”
AR	Low	“ام يه تابلطتم تقاطلا قطحمل قدعاق 5G NR؟”
FR	Low	“Comment configurer un tunnel GRE sur un routeur Huawei?”
FR	Low	“Quelle est la différence entre la fibre monomode et multimode?”
FR	Low	“Expliquez le processus de handover en 5G.”
EN	High	“50 Huawei AR6120 routers in hub-and-spoke MPLS topology, more than 90 percent CPU on hub routers during business hours, BGP 500 routes, OSPF in access layer — causes and troubleshooting?”

Bibliography & References

[1] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[2] S. Chetlur et al., "cuDNN: Efficient primitives for deep learning," *arXiv preprint*, 2014. eprint: 1410.0759.

[3] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024. eprint: 2307.08691.

[4] T. Dettmers et al., "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[5] EleutherAI, "A framework for few-shot language model evaluation," *Zenodo*, 2023. DOI: 10.5281/zenodo.10256836

[6] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, 2022.

[7] Fortune Media, *Fortune Global 500 — 2025 rankings*, Fortune Media, 2025. [Online]. Available: https://fortune.com/global500/

[8] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *International Conference on Learning Representations (ICLR)*, 2024. eprint: 2312.00752.

[9] D. Hendrycks et al., "Measuring massive multitask language understanding," in *International Conference on Learning Representations (ICLR)*, 2021. eprint: 2009.03300.

[10] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," in *Communications of the ACM*, vol. 62, 2019.

[11] C.-P. Hsieh et al., "RULER: What's the real context size of your long context language models?" In *arXiv preprint*, 2024. eprint: 2404.06654.

[12] Huawei Ascend Developer Community, "MindIE: Huawei Mind inference engine architecture and optimisation guide," Huawei Technologies, Tech. Rep., 2024. [Online]. Available: https://www.hiascend.com/en/software/mindie

[13] Huawei Ascend Developer Community, "torch_npu: PyTorch adapter for huawei ascend NPU — developer guide," Huawei Technologies, Tech. Rep., 2024. [Online]. Available: https://github.com/Ascend/pytorch

[14] Huawei Cloud, "ModelArts: AI development platform — technical documentation," Huawei Cloud, Tech. Rep., 2024. [Online]. Available: https://www.huaweicloud.com/product/modelarts.html

[15] Huawei Investment & Holding Co., Ltd., "Annual report 2024," Huawei Technologies Co., Ltd., Tech. Rep., 2024. [Online]. Available: https://www.huawei.com/en/annual-report

[16] Huawei Technologies Co., Ltd., "Ascend 910: The most powerful AI processor — technical specification," Huawei Technologies, Tech. Rep., 2019. [Online]. Available: https://e.huawei.com/en/products/servers/ascend

[17] Huawei Technologies Co., Ltd., "CANN (compute architecture for neural networks) white paper," Huawei Technologies, Tech. Rep., 2023. [Online]. Available: https://www.hiascend.com/en/software/cann

[18] International Energy Agency, "Electricity 2024: Analysis and forecast to 2026," IEA, Tech. Rep., 2024.

[19] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017.

[20] D. B. Kirk and W.-m. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 3rd. Morgan Kaufmann, 2016.

[21] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[22] J. Nickolls et al., "Scalable parallel programming with CUDA," *ACM Queue*, vol. 6, no. 2, 2008.

[23] NVIDIA, "Nemotron 3 nano: Open, efficient mixture-of-experts for mobile and edge devices," NVIDIA Corporation, Tech. Rep., 2025. [Online]. Available: https://github.com/NVIDIA-NeMo/Nemotron

[24] X. Shen et al., "HCCL: A high-performance collective communication library for distributed AI training on ascend clusters," *arXiv preprint*, 2023. eprint: 2308.00000.

[25] N. C. Thompson et al., "The computational limits of deep learning," *arXiv preprint*, 2021. eprint: 2007.05558.

Glossary

autocast Context manager for automatic mixed-precision (FP16/BF16) to reduce memory and increase speed.

chat template Formatting applied to role-labeled messages to create the token sequence for an LLM.

chunked scan An optimized Mamba scan processing input in fixed-size blocks to reduce Python loop overhead.

context parallelism Distributed strategy partitioning input sequences across device groups for attention computation.

DaVinci core The fundamental unit of Huawei Ascend NPUs, featuring specialized Cube, Vector, and Scalar units.

expert collapse A failure mode in MoE where tokens are routed to few experts, underutilizing model capacity.

GRPO Group Relative Policy Optimization — an RL algorithm using group-relative advantages for policy updates.

HCCS Huawei Cache Coherence System — high-speed chip-to-chip interconnect for Ascend NPU servers.

kernel function A function executed in parallel on a device by many threads, launched from host code.

KV-cache Memory structure storing attention keys and values to speed up incremental token decoding.

load-balancing loss Auxiliary loss in MoE models encouraging uniform token distribution across all experts.

loss mask A binary mask in SFT ensuring the model only learns from assistant tokens by zeroing others.

Megatron-Bridge NVIDIA distributed training framework that wraps Megatron-LM with unified configuration and entry points.

memory coalescing A memory access pattern where consecutive threads access consecutive addresses, merging transactions into one.

microservice An independently deployable software service focused on a single business capability, communicating via APIs.

MindIE Huawei Mind Inference Engine — an LLM serving framework for Ascend NPUs with operator fusion.

NeMo-RL NVIDIA reinforcement learning framework for LLM alignment using GRPO and Ray-based actor management.

NeMo-Run Job orchestration framework translating runspecs into Slurm jobs, managing containers and profiles.

operator fusion A compiler optimization merging multiple neural network operations into one pass to reduce memory access.

packed sequence A data format concatenating multiple sequences into one to eliminate padding and maximize hardware use.

ROUGE-L Text evaluation metric based on the longest common subsequence between hypothesis and reference.

selective scan The core recurrence operation of Mamba SSM layers, processing input-dependent state updates.

systolic array A 2D grid of processing elements where data flows rhythmically, performing MAC operations at each step.

tensor parallelism Distributed strategy partitioning weight matrices across devices to reduce partial results.

torch_npu Huawei extension registering Ascend NPUs as a native PyTorch device type.

transfer_to_npu() Utility monkey-patching CUDA calls to NPU for transparent device migration.

vLLM-Ascend Fork of vLLM with an Ascend NPU backend for high-throughput model serving.

W&B artifact A versioned data object in Weights & Biases used for tracking inter-stage pipeline handoffs.

warp A group of 32 threads in CUDA executing instructions in lock-step. The smallest unit of scheduling on an SM.